Workshops 2-5: Combined Report

Contents

Page numbers refer to this merged PDF. Click an entry to navigate.

Workshop 2: Q-Learning and SARSA 3

- Environment and algorithms 3
- Task 1: CliffWalking 4
- Task 2: Experimental design 4
- Task 2 Q1: Four-algorithm comparison 5
- Task 2 Q2: Q-learning learning rate 6
- Task 2 Q3: SARSA(lambda) trace parameter 7
- Conclusion and limits 7
- Reproducibility materials 7

Workshop 3: Deep Q-Learning 8

- Task 1 Q1: CartPole environment 8
- Task 1 Q2: DQN training 9
- Task 2 Q1: Learning rate 10
- Task 2 Q2: Exploration schedules 11
- Task 2 Q3: Replay capacity and batch size 12

Workshops 2-5: Combined Report

Contents (continued)

Page numbers refer to this merged PDF. Click an entry to navigate.

Workshop 4: Policy Gradient 13

Task 1 Q1: InvertedPendulum environment 13

Task 1 Q2: Vanilla PG, reward-to-go and baseline 14

Task 2 Q1: Learning rate 15

Task 2 Q2: Steps per batch 17

Task 2 Q3: CartPole PG versus DQN 19

Task 2 Q4: HalfCheetah controller 20

Workshop 5: Actor Critic and Soft Actor Critic 22

Task 1 Q1: HalfCheetah environment and policies 22

Task 1 Q2: AC/GAE training on HalfCheetah 23

Task 1 Q3: SAC training on HalfCheetah 24

Task 2 Q1: Effect of GAE lambda 25

Task 2 Q2: Initial entropy temperature 26

Task 2 Q3: Actor and critic learning rates 28

Task 2 Q4: Replay size, batch size and learning start 30

Supplementary temperature diagnostics 33

References and reproducibility 34

Workshop 2: Q-Learning and SARSA on CliffWalking

ELEN90098 Reinforcement Learning for Engineering
Group: Zhengnan Cao and Yuchen Yang

Abstract

We implemented tabular Q-learning, SARSA(0), five-step SARSA, and SARSA(λ) and evaluated them in Gymnasium's CliffWalking-v1. Task 1 established the environment, completed the provided code, and compared one 1,000-episode training run per algorithm in TensorBoard. Task 2 repeated the comparison with five random seeds and varied Q-learning's learning rate α and SARSA(λ)'s trace parameter λ. Q-learning consistently finished with a return of −13, the shortest safe route. SARSA(0) and SARSA(λ=0.1) finished near −17, while five-step SARSA varied more across seeds. With our operational speed measure, SARSA(λ=0.1) reached a return of −25 earliest. All tested α values eventually achieved −13; moderate λ values learned faster than λ=0 or λ=0.9 under the fixed training schedule.

1. Environment and algorithms

CliffWalking is a 4 × 12 grid. The agent starts at (3, 0), aims for (3, 11), and the ten cells (3, 1) through (3, 10) are a cliff. An ordinary move, including the move into the goal, gives reward −1. Entering the cliff gives −100 and returns the agent to the start without ending the episode; entering the goal ends the episode. Actions are discrete: 0 = up, 1 = right, 2 = down, and 3 = left. The environment is finite and discrete. There are 37 nonterminal decision positions (the upper 36 cells and the start). Gymnasium declares Discrete(48) because it encodes all 4 × 12 grid positions; the goal may also appear as a terminal observation, while cliff positions are not occupied after a step.

The agent uses an ε-greedy behavior policy. For a transition (s, a, r, s'), the tabular update is $Q(s,a) \leftarrow Q(s,a) + \alpha\delta$. The algorithms differ in the target used to calculate the temporal-difference error δ:

Algorithm	Target for a nonterminal transition	Key property
Q-learning	$r + \gamma \max_{a'} Q(s',a')$	Off-policy; evaluates a greedy continuation even when the behavior explores.
SARSA(0)	$r + \gamma Q(s',a')$	On-policy; uses the next action actually selected by the ε-greedy policy.
SARSA(n), n=5	$\sum_{i=0}^{n-1} \gamma^i r_{t+i+1} + \gamma^n Q(s_{t+n},a_{t+n})$	Propagates information through five rewards before bootstrapping; the terminal remainder has no bootstrap term.
SARSA(λ)	One-step SARSA error applied to eligible earlier state–action pairs	Uses accumulating traces: $e(s,a) \leftarrow e(s,a)+1$ for the visited pair, then $Q \leftarrow Q+\alpha\delta e$ and $e \leftarrow \gamma\lambda e$.

For a terminal transition, the bootstrap term is zero. The learning discount was $\gamma=0.99$. The reported episode return is the undiscounted sum of rewards, which is distinct from the discounted target used in learning. A direct safe route requires 13 moves, so its undiscounted return is −13.

2. Task 1: CliffWalking

Question 1: State, action, and reward spaces

The finite state and action spaces and the reward and termination rules are described in Section 1. The observation is an integer row × 12 + column; the starting observation is 36 and the goal is 47. Rewards in this task are discrete, with −1 for an ordinary move and −100 for a cliff entry.

Question 2: Initial comparison

We completed the TODO sections in the supplied agent and training script, trained Q-learning, SARSA(0), SARSA(n=5), and SARSA(λ=0.1) for 1,000 episodes each, and compared the TensorBoard episode-return curves.

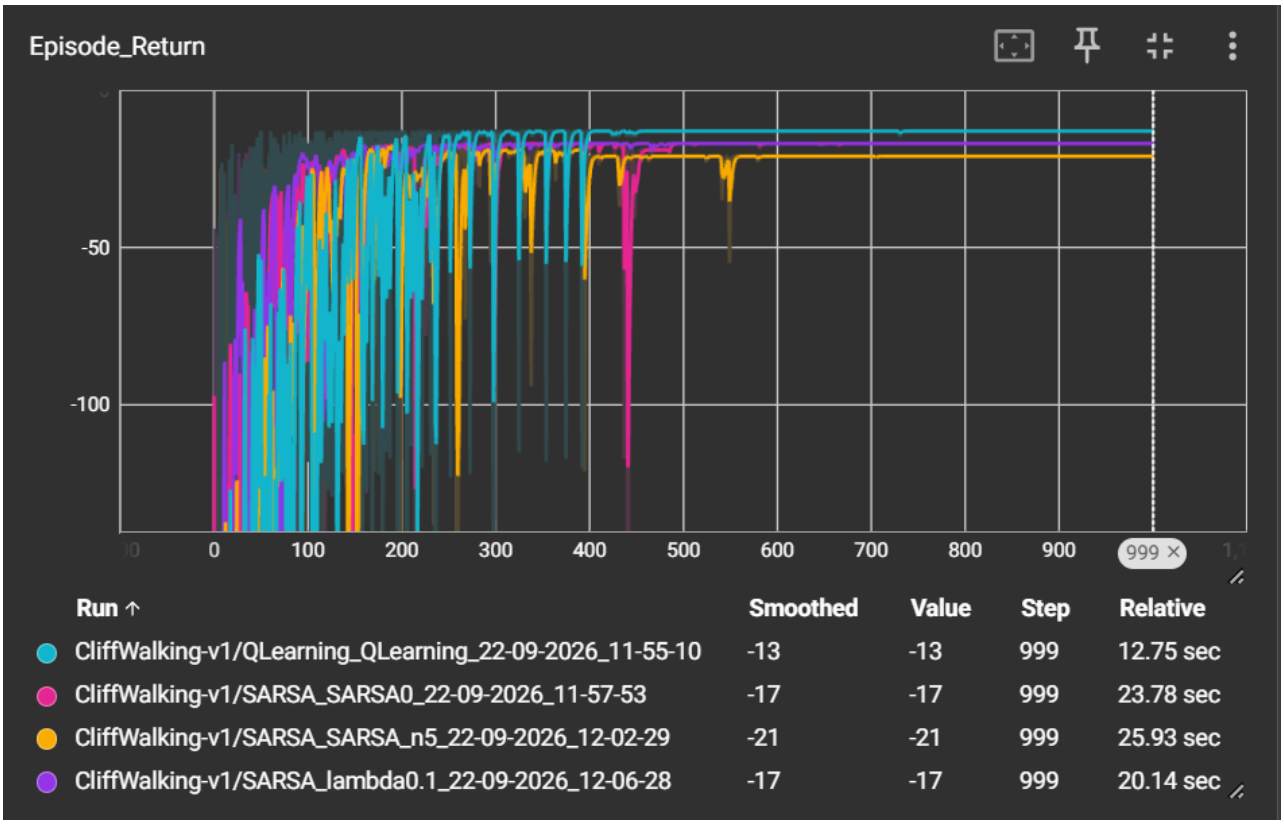


Figure 1. The four Task 1 runs. Values at the final logged step are −13, −17, −21, and −17 for Q-learning, SARSA(0), SARSA(n=5), and SARSA(λ=0.1), respectively. This is a single-run illustration rather than a multi-seed estimate.

All four runs improved sharply from large negative returns caused by exploration and cliff falls. Q-learning reached the shortest safe-route return of −13. The three on-policy runs ended with longer routes in this initial comparison. The Task 2 repetitions below quantify how much of this comparison persists across random seeds; in particular, five-step SARSA's final result varies between runs.

3. Task 2: Experimental design

We used the completed Task 1 QLAgent on CliffWalking-v1. Each distinct configuration was trained for **1,000 episodes with each of five seeds, 0–4**. The common settings were $\gamma=0.99$, initial $\varepsilon=0.5$, episode-wise decay factor 0.99, minimum $\varepsilon=0$, initial Q value 0, and baseline $\alpha=0.5$. The comparisons were:

1. Q-learning, SARSA(0), SARSA(n=5), and SARSA(λ=0.1).
2. Q-learning with $\alpha \in \{0.1, 0.3, 0.5, 0.9\}$.

3. SARSA(λ) with $\lambda \in \{0, 0.1, 0.5, 0.9\}$; $\lambda=0$ is SARSA(0).

Shared settings were run once and reused in the relevant comparisons, giving **nine distinct configurations, 45 runs, and 45,000 training episodes**. A 2,000-step safety cap bounded unusually long episodes; capped episodes were retained in the training results. For each run, we recorded every episode's undiscounted return, the mean of the last 100 training returns, and one final greedy evaluation without exploration or learning. Every final greedy evaluation reached the goal.

To compare learning speed, we report the **first episode at which a 50-episode rolling mean was at least −25 for 20 consecutive rolling windows**. The table gives the median of this episode number across five seeds. This fixed-threshold statistic measures attainment of a useful return; it is not a proof of asymptotic convergence. Figures show the mean of the five smoothed runs with a shaded band of ±1 sample standard deviation.

4. Task 2, Question 1: Four-algorithm comparison

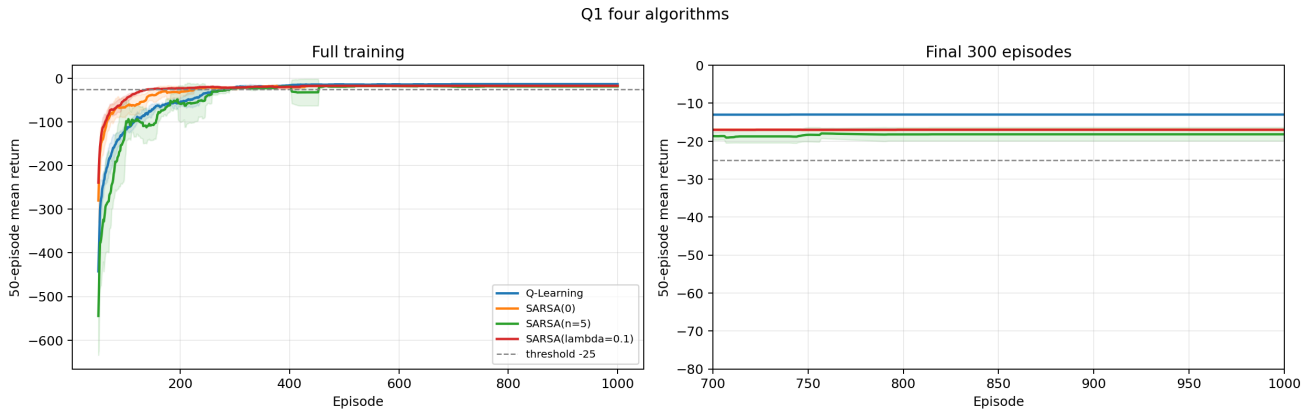


Figure 2. Mean 50-episode rolling returns over five seeds (shading: ±1 standard deviation). The right panel enlarges the final 300 episodes.

Algorithm	Mean return, first 100 episodes	Last-100 return, mean ± SD	Final greedy return, mean ± SD	Median episode reaching −25
Q-learning	−280.0	−13.00 ± 0.00	−13.00 ± 0.00	275
SARSA(0)	−172.2	−17.01 ± 0.02	−17.00 ± 0.00	224
SARSA(n=5)	−326.3	−18.20 ± 1.79	−18.20 ± 1.79	277
SARSA(λ=0.1)	−143.5	−17.00 ± 0.01	−17.00 ± 0.00	**141**

SARSA(λ=0.1) attained the −25 threshold first, followed by SARSA(0). Q-learning ultimately performed best: every seed obtained −13 in both the last 100 training episodes and greedy evaluation. SARSA(0) and SARSA(λ=0.1) finished around −17. Five-step SARSA finished at −17, −21, −17, −19, and approximately −17 across the five seeds, accounting for its larger final variability and for the −21 endpoint in the single Task 1 run.

This behavior is consistent with the update targets. Q-learning bootstraps from the greedy next action and can learn the short route beside the cliff despite exploratory behavior. SARSA methods account for the next action the behavior policy may actually take, so cliff risk during exploration can favor a longer route. Eligibility traces can spread useful feedback to earlier decisions, plausibly explaining the earlier threshold crossing for λ=0.1. Five-step SARSA must wait for several rewards before an update and showed more variation in these runs. These are interpretations of the observed runs, not claims that one method is

always faster. Because ε decays rapidly toward zero, a safe on-policy route learned early can persist through the finite training period.

Five of 5,000 five-step SARSA training episodes reached the 2,000-step cap. They affect early-return averages but none of the five final greedy policies failed to reach the goal.

5. Task 2, Question 2: Q-learning learning rate α

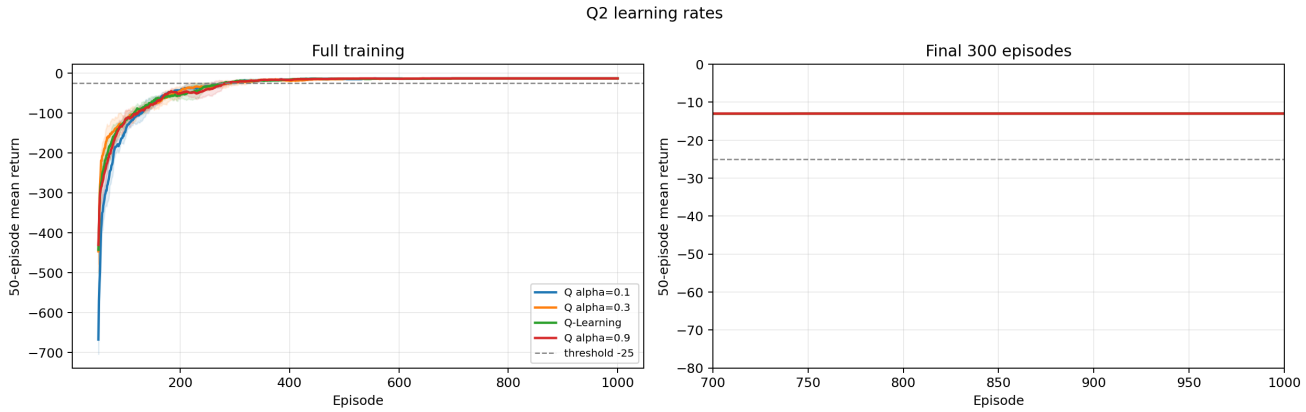


Figure 3. Q-learning with four learning rates; all other settings and seeds are held fixed.

α	Mean return, first 100 episodes	Last-100 return, mean ± SD	Final greedy return	Median episode reaching −25
0.1	−404.9	−13.00 ± 0.00	−13.00	287
0.3	−281.3	−13.00 ± 0.00	−13.00	275
0.5	−280.0	−13.00 ± 0.00	−13.00	275
0.9	−272.4	−13.00 ± 0.00	−13.00	282

A smaller α incorporates each new TD error more slowly, and α=0.1 had the weakest average return in the first 100 episodes. Raising α can transmit new information faster, but the threshold times here are **not monotonic**: α=0.9 had a slightly better early mean than α=0.5, yet its median threshold episode was 282 rather than 275. A large α also gives individual cliff penalties more influence and may increase sensitivity to recent experience; these data do not establish a persistent high-α instability. After 1,000 episodes, all four tested rates yielded the same −13 final policy in all five seeds. Thus α mainly affected the learning transient within this tested range and schedule, rather than the final outcome.

6. Task 2, Question 3: SARSA(λ) trace parameter

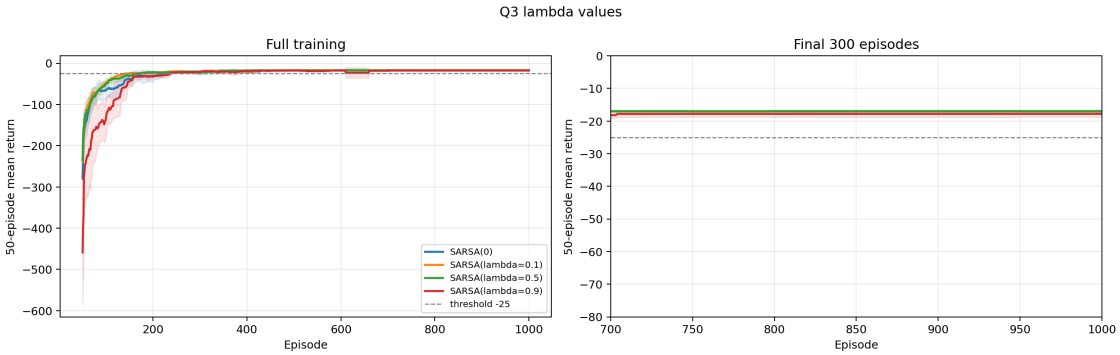


Figure 4. SARSA(λ) with four trace values; λ=0 coincides with SARSA(0).

λ	Mean return, first 100 episodes	Last-100 return, mean ± SD	Final greedy return, mean ± SD	Median episode reaching −25
0	−172.2	−17.01 ± 0.02	−17.00 ± 0.00	224
0.1	−143.5	−17.00 ± 0.01	−17.00 ± 0.00	**141**
0.5	−142.6	−17.00 ± 0.00	−17.00 ± 0.00	171
0.9	−294.8	−17.80 ± 1.09	−17.80 ± 1.10	219

Moderate traces improved learning speed relative to λ=0. Among these values, λ=0.1 reached the threshold earliest, while λ=0.5 also performed well; both finished at −17 in every greedy evaluation. λ=0.9 was slower early and finished at −19 in two seeds and −17 in three. Two of its 5,000 training episodes reached the safety cap. With accumulating traces, a larger λ keeps credit assigned to more previous state–action pairs. This can accelerate propagation when rewards are informative, but it can also spread large cliff penalties or exploratory TD errors over a longer history. The observed λ=0.9 variability is consistent with that trade-off. The best λ depends on the environment, α, ε schedule, and training budget; the present experiment identifies 0.1 as the fastest of the tested values under these conditions.

7. Conclusion and limits

The experiments separate speed of obtaining a useful return from final path quality. SARSA(λ=0.1) first met the −25 threshold, whereas Q-learning alone consistently reached the shortest safe-route return of −13. Changing Q-learning's α altered early learning but not its final result over the tested values. Moderate λ improved SARSA's learning speed, while λ=0.9 gave less reliable final routes in this sample.

The comparison uses five seeds, one deterministic gridworld, a finite 1,000-episode budget, and a rapidly decaying ε. The ±SD bands describe observed variation across runs, not confidence intervals or statistical significance. The threshold is an operational measure, and rare capped episodes make some early averages more negative. A different exploration schedule, training budget, or environment could change the ordering.

Reproducibility materials

- Executed notebook
- Task 2 results archive: per-episode returns, per-run metrics, summaries, and figures
- Task 1 answer and original figure
- Workshop instructions

Workshop 3: Deep Q-Learning - Final Report

Authors: Zhengnan Cao, Yuchen Yang

This report answers Task 1 and all three Task 2 questions using the audited experiment set. Each condition uses three training seeds (0, 1, 2), 200,000 training steps, and 10 greedy evaluation episodes per checkpoint. Evaluation occurs every 10,000 training steps using reset seeds 10000-10009. Evaluation steps are excluded from the horizontal axis. Curves show the mean and one sample standard deviation across training seeds, without smoothing.

Tables report mean ± SD. Last-three return averages three final checkpoints per seed. Threshold time confirms three consecutive evaluation means of at least 475; times are medians among successful seeds and counts show how many reached it. Confirmation does not guarantee later stability. Reference: provided ELEN90098 Workshop 3 handout. Three seeds give descriptive results, not a universal ranking.

Task 1 - Question 1: CartPole environment

Question

What are the state space, action space and reward structure of CartPole? Are they finite or infinite, discrete or continuous? Is the environment deterministic or stochastic?

Answer

Item	Answer
State	Four real-valued variables: cart position, cart velocity, pole angle and pole angular velocity. The state space is continuous and infinite.
Action	Two actions: 0 pushes left and 1 pushes right. The action space is discrete and finite.
Reward	+1 per step, including the first failure step. Immediate reward is discrete and finite. CartPole-v1 allows at most 500 steps, so the maximum episode return is 500.
Dynamics	Deterministic given a state and action. Reset adds random initial-state perturbations, so episodes can still differ.

The episode terminates when cart position leaves approximately ±2.4 m or pole angle leaves approximately ±12 degrees. The 500-step time limit truncates an otherwise continuing episode.

Task 1 - Question 2: DQN training

Question

Train DQN on CartPole-v1 and check whether it can achieve the maximum return of 500. Use TensorBoard, submit evaluation returns versus environment steps, and briefly comment on training success.

Submit

The required evaluation curve is shown below. This uses the learning-rate 0.001 runs from the corrected experiment set, rather than the historical Task 1 run.

Configuration: three hidden layers of 64 units, learning rate 0.001, discount 0.99, replay capacity 100,000, batch size 64, and target update every 500 gradient updates. Learning starts after 1,000 steps. Epsilon starts at 1, decays by 0.995 after each episode and has a minimum of 0.02.

The DQN target is:

$$y = r + \gamma(1 - d)\max_{a'} Q_{\text{target}}(s', a')$$

Here d is 1 for true environment termination and 0 otherwise; a time-limit truncation still permits bootstrapping.

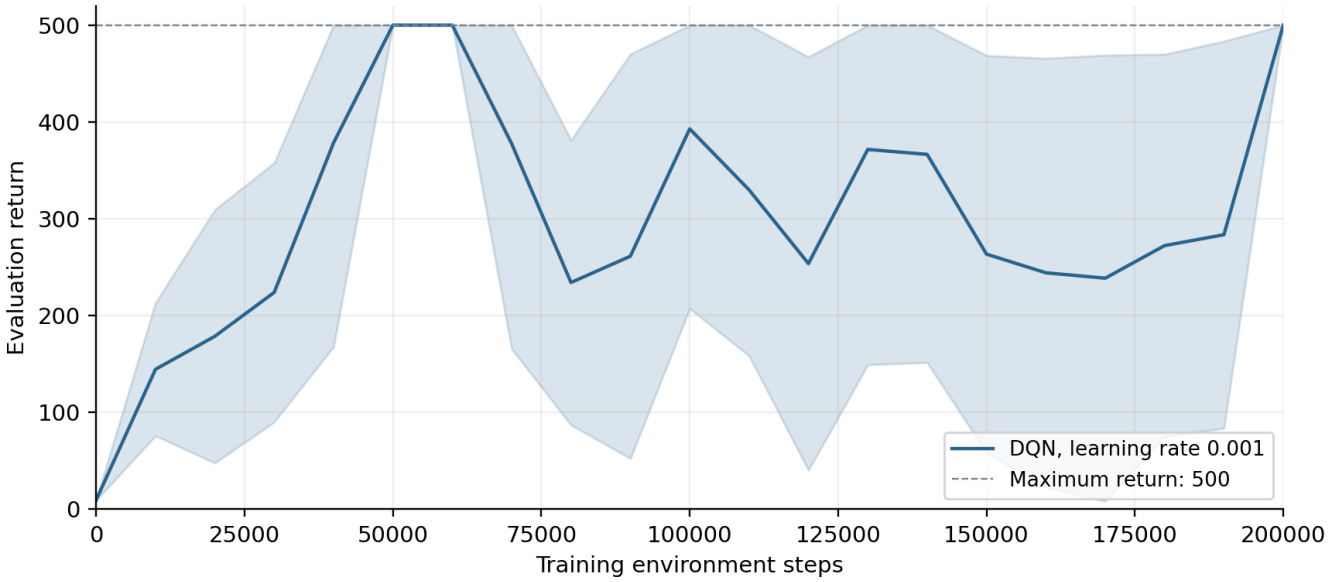


Figure 1. DQN evaluation returns versus training environment steps.

Discussion

Does the curve show successful training?

All three seeds finish with a 10-episode evaluation mean of 500, so the agent achieves the required maximum. However, the average of the last three checkpoints is 351.9 ± 82.1, showing earlier late-stage drops; this is successful maximum-return performance, not stable convergence throughout training.

TensorBoard scalar exports of these saved evaluations are provided in the separate verification material. The submitted curve uses the same evaluation records.

Task 2 - Question 1: Learning rate

Question

Investigate how learning rate affects training by comparing at least three values. Here the rates are 0.0001, 0.001 and 0.01, with all other settings fixed as above.

Submit

Submit evaluation returns versus environment steps for each learning rate.

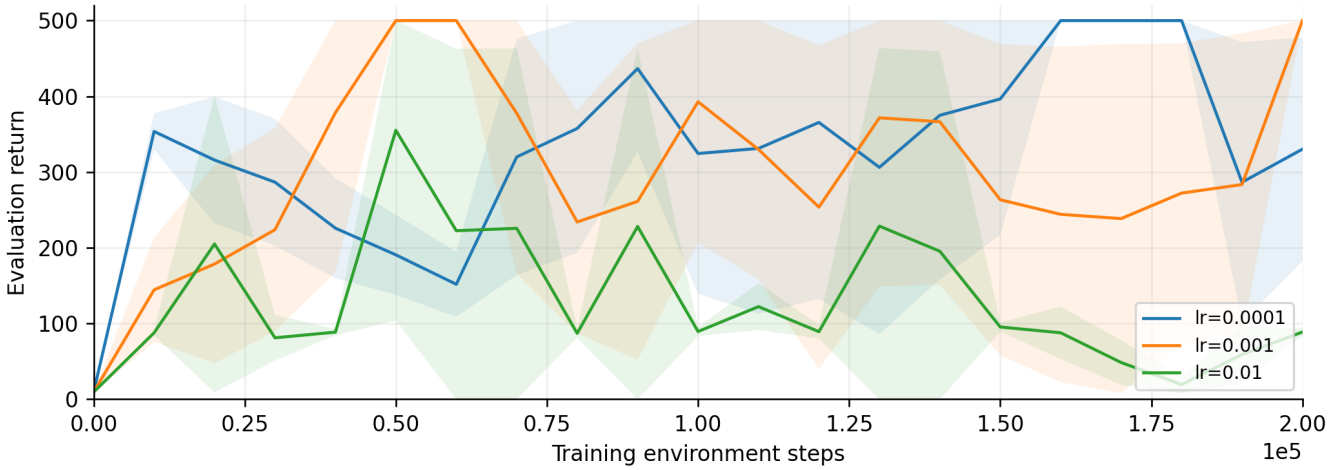


Figure 2. Learning-rate comparison.

Setting	Final return	Last-three return	Steps to threshold (seeds)
0.0001	330.2 ± 147.6	372.2 ± 110.7	150,000 (3/3)
0.001	500.0 ± 0.0	351.9 ± 82.1	60,000 (3/3)
0.01	88.7 ± 4.5	55.7 ± 7.8	70,000 (1/3)

Discussion

Which learning rate achieves faster convergence?

Learning rate 0.001 confirms the threshold fastest across all three seeds: median 60,000 steps, compared with 150,000 for 0.0001. Rate 0.01 confirms in only one of three seeds.

Which learning rate is more stable?

Among the two useful rates, 0.0001 has smaller late-training fluctuations: mean temporal SD over the last five checkpoints is 95.0, versus 180.4 for 0.001. Its final return is lower, so stability and endpoint performance are different criteria.

Does too high a learning rate cause divergence or unstable oscillations?

At 0.01 the curves fluctuate and final return is only 88.7 ± 4.5. This indicates poor and unstable learning here; finite losses do not establish numerical divergence.

Does too low a learning rate cause slow learning or poor final performance?

Rate 0.0001 takes longer to confirm near-maximum performance and ends below 0.001. It still learns effectively and is more stable late in training, so a smaller rate is not simply ineffective.

Task 2 - Question 2: Exploration schedules

Question

Compare constant epsilon 0.1 with two episode-based decays: multiply epsilon by 0.9 or 0.995 after each completed episode, starting from 1 and stopping at a minimum of 0.02.

Submit

Submit evaluation returns versus environment steps for all three schedules.

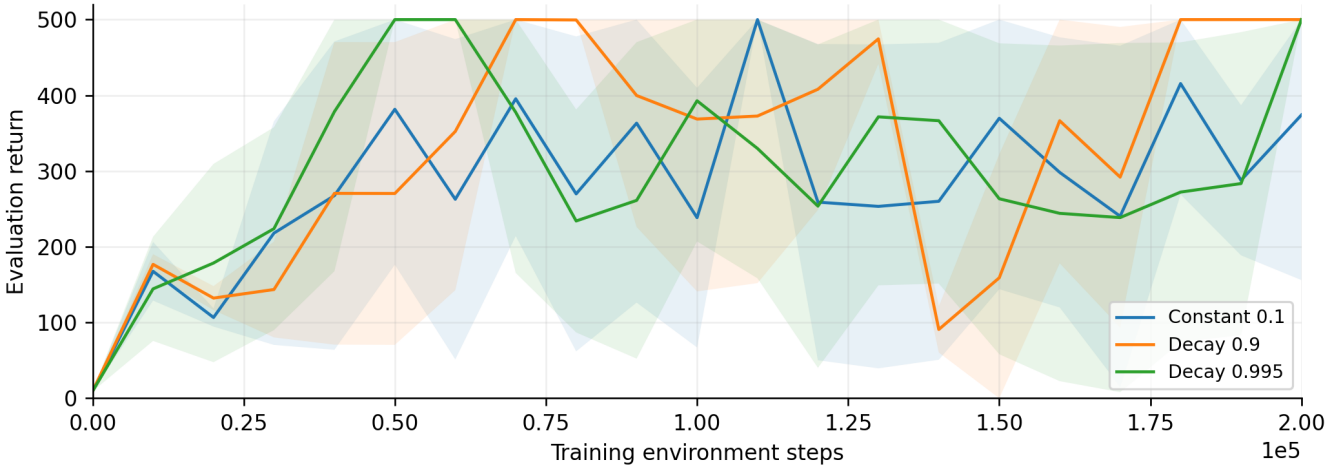


Figure 3. Exploration-schedule comparison.

Setting	Final return	Last-three return	Steps to threshold (seeds)
Constant 0.1	374.1 ± 218.1	359.1 ± 32.3	80,000 (2/3)
Decay 0.9, floor 0.02	500.0 ± 0.0	500.0 ± 0.0	90,000 (3/3)
Decay 0.995, floor 0.02	500.0 ± 0.0	351.9 ± 82.1	60,000 (3/3)

Discussion

How do the schedules affect exploration and learning?

Constant epsilon maintains a 10% random-action probability. The 0.9 schedule reaches its floor at 399-470 steps; the 0.995 schedule reaches it at 114,641-143,081 steps. The fast schedule therefore becomes mostly greedy before the 1,000-step learning start, yet gives the best last-three return here.

Does keeping epsilon higher for longer improve performance or unnecessarily slow learning?

Longer exploration does not improve late performance in these runs: the last-three means are 500.0 for fast decay, 351.9 for slow decay and 359.1 for constant epsilon. Slow decay confirms the near-maximum threshold earlier, however, so this is not a universal speed ranking.

What happens if epsilon is too small or decays too quickly?

In general, too little exploration can miss useful actions. That failure is not demonstrated here: fast decay performs well on CartPole; state-space coverage was not measured, so the curves alone cannot explain why.

Task 2 - Question 3: Replay capacity and batch size

Question

Compare replay capacities 1,000, 10,000 and 100,000, and batch sizes 32 and 128. All six combinations use the same other settings.

Submit

Submit evaluation returns versus environment steps for all cases. In the table, the setting is replay capacity / batch size.

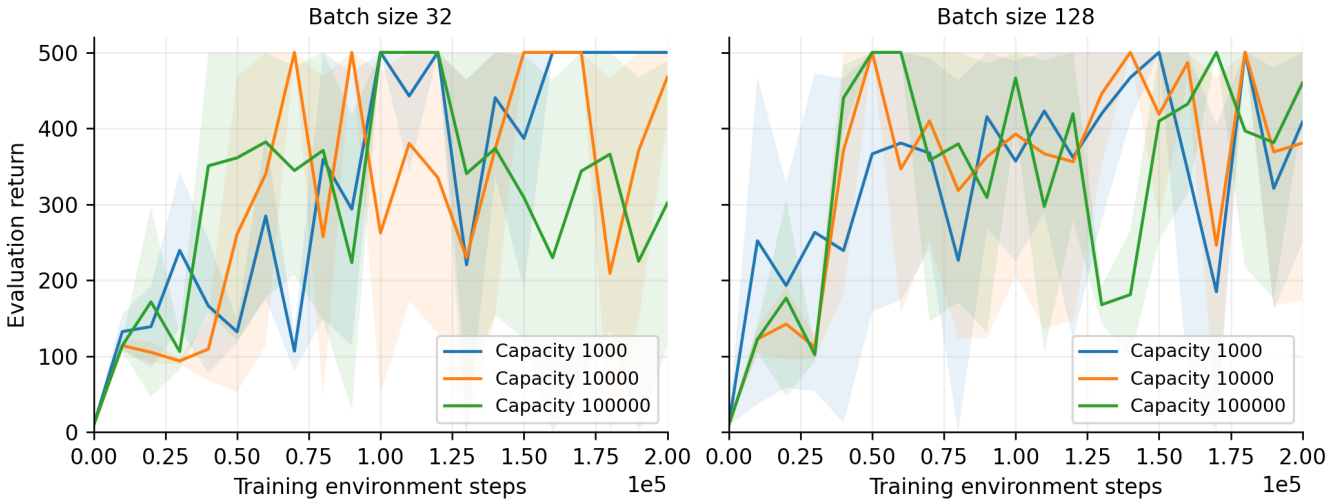


Figure 4. Replay-capacity and batch-size comparison.

Setting	Final return	Last-three return	Steps to threshold (seeds)
1000 / 32	500.0 ± 0.0	500.0 ± 0.0	120,000 (3/3)
1000 / 128	408.3 ± 158.9	409.8 ± 71.8	130,000 (3/3)
10000 / 32	467.5 ± 56.3	349.1 ± 162.6	110,000 (3/3)
10000 / 128	380.6 ± 206.8	416.5 ± 63.7	100,000 (3/3)
100000 / 32	301.6 ± 186.9	297.5 ± 60.0	120,000 (3/3)
100000 / 128	459.7 ± 40.4	412.5 ± 67.6	180,000 (3/3)

Discussion

How does memory size change curve stability and smoothness?

The effect depends on batch size. With batch 32, capacity 1,000 is steadier late in training than 100,000 (temporal SD 0.0 versus 185.7); with batch 128, increasing capacity reduces late temporal SD from 166.2 to 113.7.

How does batch size affect convergence speed and final performance?

Batch 128 does not always improve performance. At capacity 1,000 it lowers last-three return from 500.0 to 409.8; at 10,000 it raises it from 349.1 to 416.5 and confirms the threshold 10,000 steps earlier; at 100,000 it raises return but confirms 60,000 steps later.

Which combination gives the best CartPole performance in these experiments?

Capacity 1,000 with batch 32 is best by last-three return: 500.0 ± 0.0. This is the clearest late-performance result within the tested budget, not a claim that it is best for every task or performance measure.

Workshop 4: Policy Gradient - Final Report

Authors: Zhengnan Cao, Yuchen Yang

This report combines Task 1 and Task 2 using the corrected PG experiments. Unless stated otherwise, results use training seeds 0, 1 and 2; each checkpoint averages ten stochastic evaluation episodes with fixed seeds 10000-10009, isolated from training. Curves show the training-seed mean and one sample standard deviation, not a confidence interval. Evaluation steps are excluded from training budgets.

Task 1 - Question 1: InvertedPendulum environment

Question

What are the state space, action space and rewards? Are they finite or infinite, discrete or continuous? Is the environment deterministic or stochastic?

Answer

Component	Description and classification
State / observation	Cart position, pole angle, cart velocity and pole angular velocity; four continuous coordinates, with an unbounded observation Box and infinitely many possible values.
Action	One continuous cart-force command in [-3, 3]; bounded but with infinitely many possible values.
Reward	A finite, discrete reward: 1 after a healthy transition, 0 on a terminating transition. The 1000-step time limit bounds episode return by 1000.

In the installed Gymnasium 1.3.0 implementation, an observation containing non-finite values or a pole angle with absolute value greater than 0.2 radians terminates the episode; the TimeLimit wrapper truncates at 1000 steps. Given the full simulator state and action, physics transitions are deterministic. Resets add uniform random noise, so episodes have stochastic initial states; sampling a stochastic policy introduces additional action randomness.

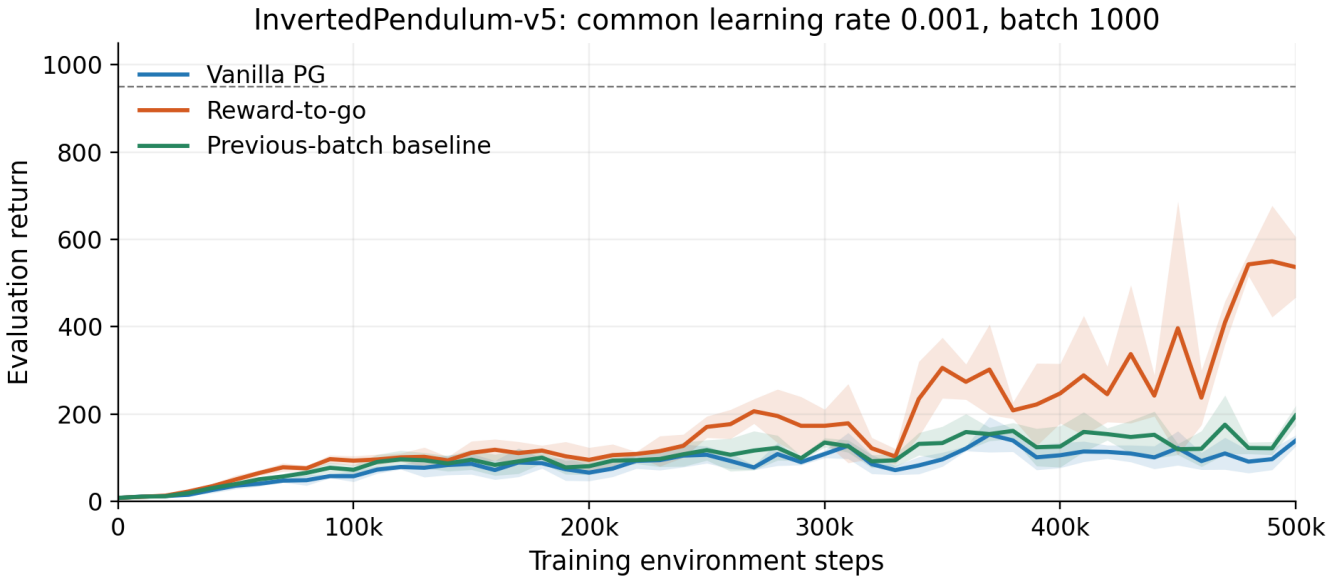
Task 1 - Question 2: Vanilla PG, reward-to-go and baseline

Question

Run the three PG methods on InvertedPendulum, visualize progress with TensorBoard, and assess training success and policy randomness.

Submit

The required evaluation-return versus environment-step plot compares all three methods at learning rate 0.001, minimum batch 1000, gamma 0.99 and a 64 x 2 ReLU network. It reuses the nine corresponding corrected runs from Task 2, rather than treating them as additional experiments. Each run requests 500,000 training steps; final saved evaluations may slightly exceed this after a complete batch, while the plot ends at 500,000.



Task 1: comparison of the three PG methods

TensorBoard logging was enabled for these runs; the figure reconstructs the evaluation scalars from their CSV records. Event-file verification does not constitute a browser inspection.

Discussion

Does the learning curve indicate successful training?

All methods improve, with final evaluation returns of 141.5 ± 13.6 for Vanilla, 536.3 ± 69.3 for RTG and 201.8 ± 20.9 for baseline. None achieves three consecutive evaluation means of at least 950, so these settings show learning but do not establish that the task is solved.

Is the policy deterministic or stochastic?

It is stochastic: a Gaussian actor samples actions during both training and evaluation; only the environment-executed action is clipped to its bounds. Fixing evaluation seeds makes those sampled episodes reproducible without making the policy deterministic.

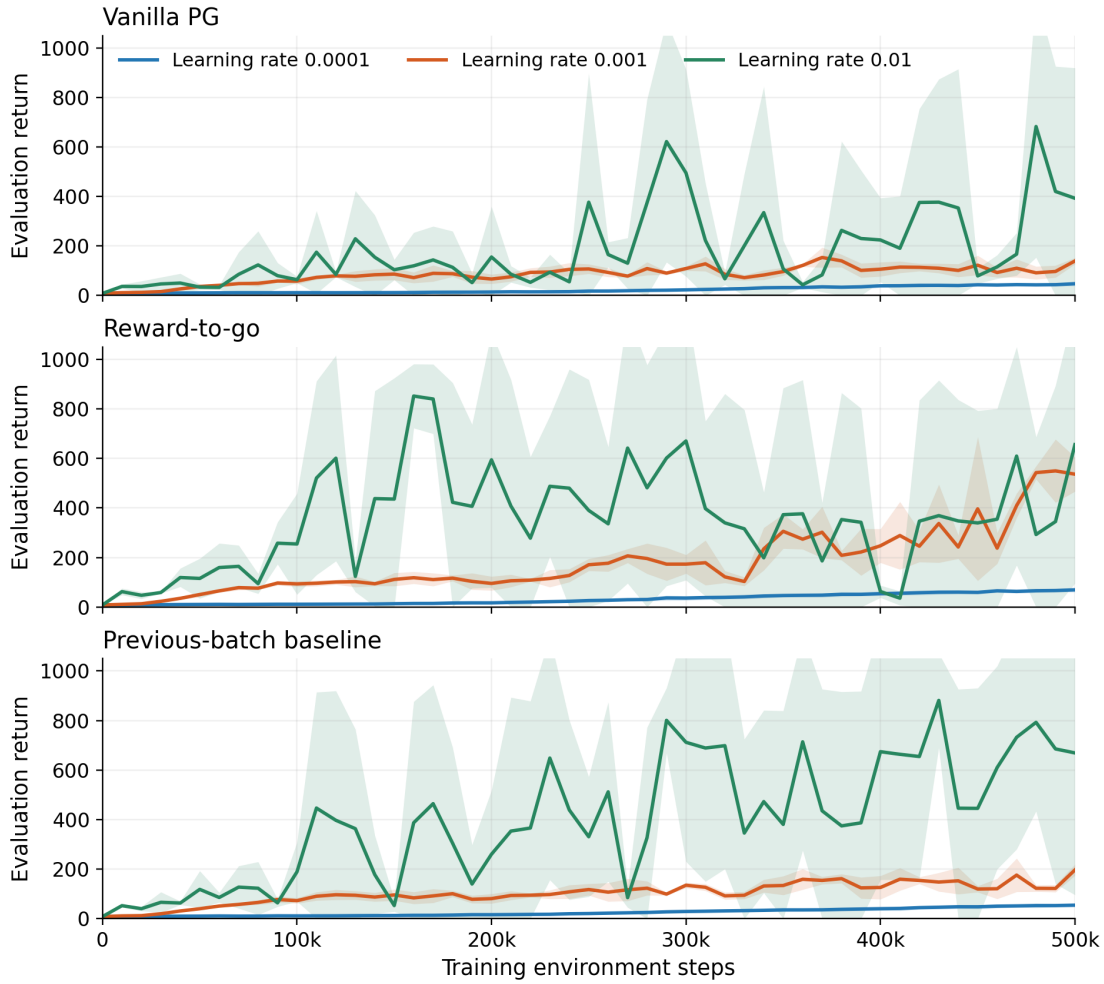
Task 2 - Question 1: Learning rate

Question

Investigate at least three learning rates α in [0.00001, 0.01], including their interaction with reward-to-go and baseline.

Submit

The three panels plot evaluation return against environment steps for learning rates 0.0001, 0.001 and 0.01. Minimum batch is 1000; the other Task 1 settings remain fixed.



Learning rates for Vanilla, RTG and previous-batch baseline

Each cell is the across-seed mean ± SD of the average return over the final five evaluation checkpoints.

Method / LR	0.0001	0.001	0.01
Vanilla	43.9 ± 5.9	106.1 ± 19.9	351.7 ± 305.2
RTG	66.2 ± 7.6	457.7 ± 45.8	452.6 ± 438.1
Baseline	52.4 ± 7.1	149.1 ± 24.0	700.7 ± 465.2

RTG includes the outer gamma-to-the-time-step factor, keeping discount powers relative to episode origin. Baseline is the mean discounted trajectory return from previous batches only, initially zero; it is not a same-batch mean or a learned value function. Core runs use Adam, with no advantage normalization, GAE or gradient clipping.

Discussion

Which learning rate achieves the fastest convergence?

Using three consecutive evaluation means of at least 950, learning rate 0.01 is the fastest successful setting: RTG reaches this criterion in 2/3 seeds, with a success-conditioned median of 225,451 steps; baseline reaches it in 2/3 at 320,296.5 steps. Vanilla reaches it in only 1/3 at 500,249 steps, beyond the nominal 500,000-step budget; lower rates never reach it, and confirmation does not guarantee later stability.

How does learning rate interact with RTG and baseline: does a higher rate amplify or mitigate their effect?

Increasing the rate from 0.001 to 0.01 reduces RTG's last-five return advantage over Vanilla from 351.6 to 100.8, while increasing baseline's advantage from 43.0 to 349.0. Thus the observed interaction depends on the method: a higher rate does not uniformly strengthen variance-reduction benefits, and its large seed variation prevents a reliable universal ranking.

Is this related to gradient-estimate variance: can a large rate amplify noisy updates or speed learning when variance is reduced?

A larger rate can magnify noisy parameter changes, consistent with the irregular 0.01 curves and large across-seed spread, while also producing earlier threshold crossings. The fixed-policy diagnostic in Question 2 supports lower local gradient variance for RTG and baseline, but does not measure variance along the changing training policies or prove that noise alone caused these learning differences; Adam also rescales updates.

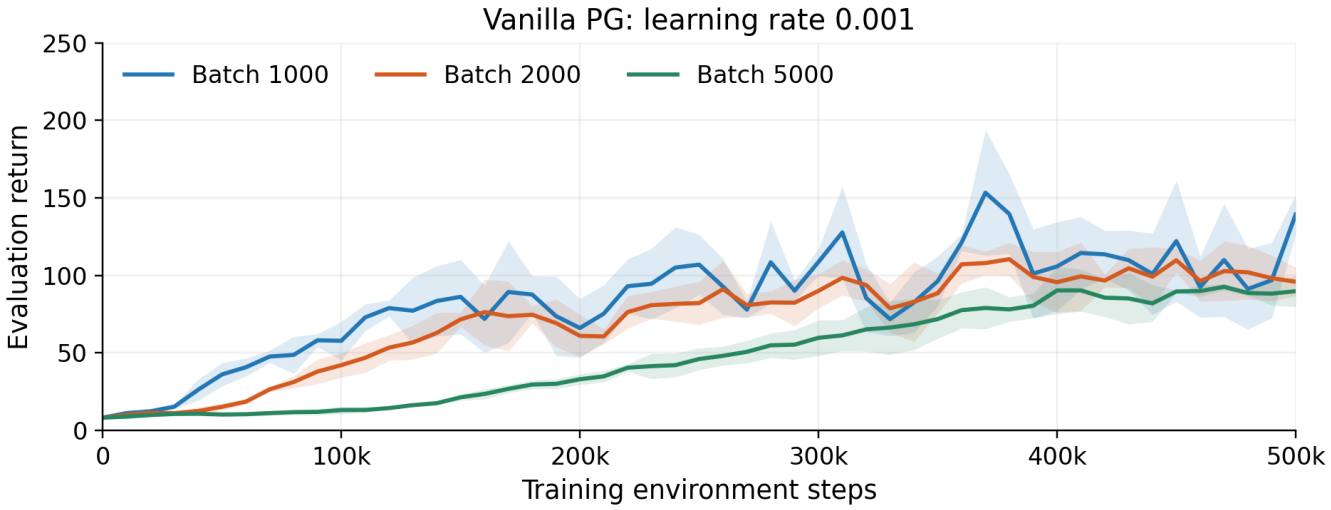
Task 2 - Question 2: Steps per batch

Question

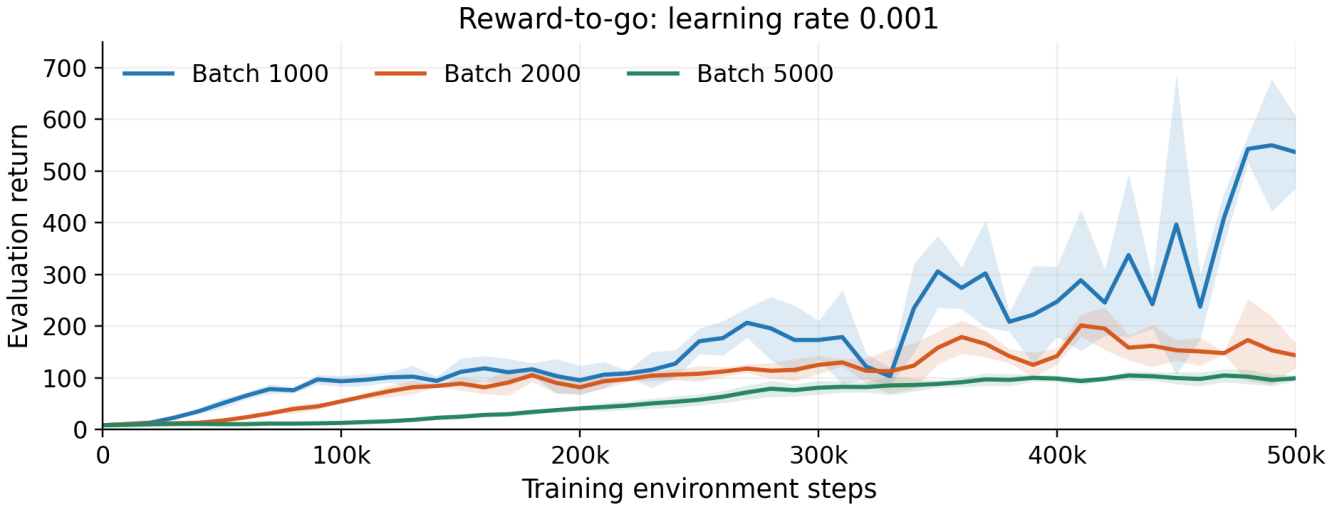
Compare at least three batch-step settings for Vanilla PG, reward-to-go and baseline, and assess return, sample efficiency and gradient variance.

Submit

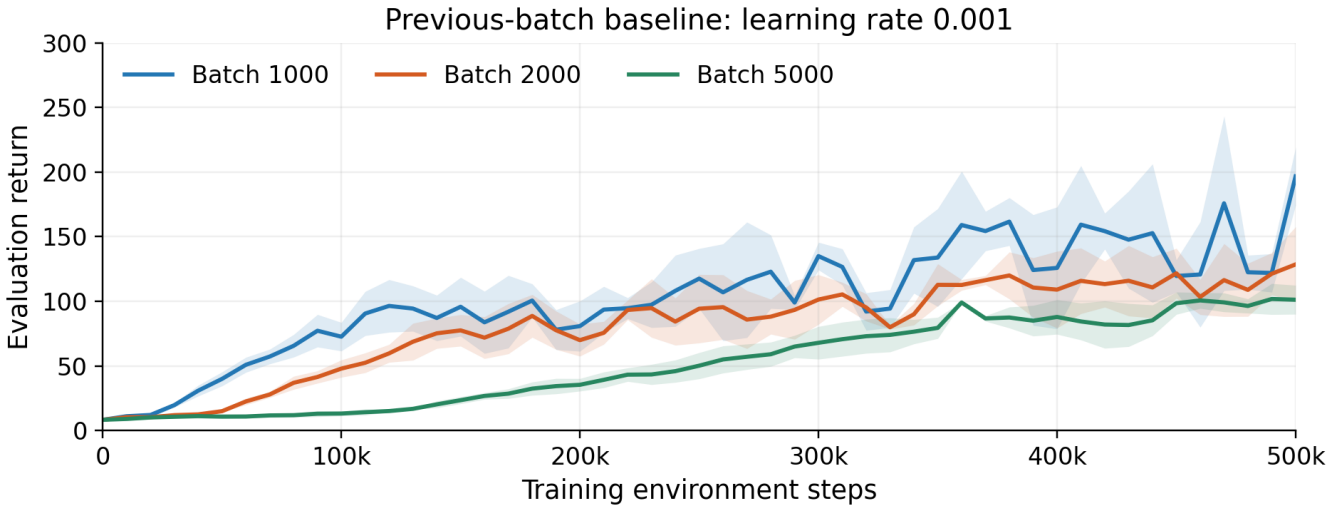
The required three figures each compare minimum batch sizes 1000, 2000 and 5000 at learning rate 0.001. Each run has the same 500,000-step requested budget and finishes complete episodes; actual totals may slightly exceed that budget. Tables use final saved evaluations, while curve integration ends at the requested budget.



Vanilla PG: three batch sizes



Reward-to-go: three batch sizes



Previous-batch baseline: three batch sizes

Discussion

For fixed environment steps, does increasing batch steps consistently improve final evaluation return?

No: mean final returns decrease as batch grows from 1000 to 2000 to 5000: Vanilla 141.5 → 95.9 → 90.2, RTG 536.3 → 141.9 → 101.4, and baseline 201.8 → 130.1 → 100.8. More samples per update therefore do not improve final return in these tested settings.

How does the trade-off between lower gradient variance and more computation per update appear in sample efficiency?

From batch 1000 to 5000, mean update counts fall from 474.7, 450.3 and 469.0 to 100 for Vanilla, RTG and baseline. Their budget-normalized AUC returns also fall, respectively, from 84.1 to 47.8, 177.6 to 57.2 and 102.2 to 51.8. Larger batches show lower local diagnostic variance but lower AUC; fewer updates may contribute, although these measurements do not isolate the cause. Concurrent wall times do not establish computational-speed rankings.

Does a larger batch reduce variance as strongly as in Vanilla, and which variance-reduction technique benefits least?

Define benefit as the proportional reduction in gradient-covariance trace from batch 1000 to 5000. Both techniques show a stronger reduction than Vanilla at the tested frozen policy, but **baseline has the smaller benefit of the two techniques**: 74.0%, versus RTG's 76.2%.

Estimator	Trace at batch 1000	Trace at batch 5000	Reduction
Vanilla	0.4381	0.1286	70.7%
RTG	0.0840	0.0200	76.2%
Baseline	0.1162	0.0302	74.0%

This separate diagnostic uses 20 independent paired collections at one frozen initial policy; batch sizes are nested complete-episode prefixes. Baseline is fixed from an independent 5003-step calibration batch, outside formal training budgets. Results are conditional on that policy and calibration value; finite replicates, random batch lengths and step averaging limit a general or strictly unbiased interpretation, and do not prove the same ranking throughout learning.

Task 2 - Question 3: CartPole PG versus DQN

Question

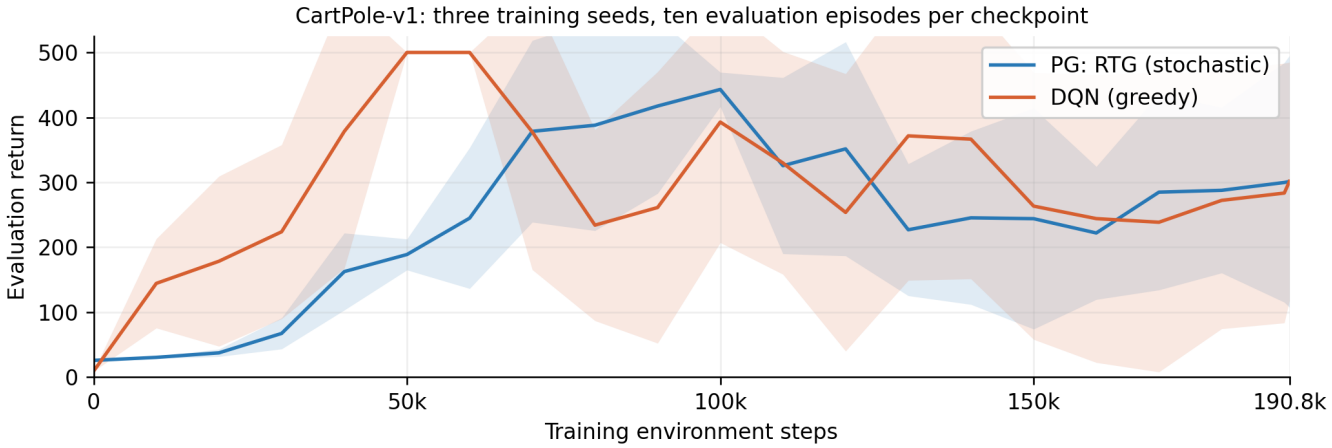
Run PG on CartPole, tune its training budget, batch steps, learning rate, gamma and network size, and compare its learning speed with DQN.

Submit

Both methods use three seeds, a 200,000-step requested budget, 64 x 3 ReLU networks and gamma 0.99. PG is newly trained; DQN reuses the WS3 baseline. Seed-0 pilots selected PG's method, learning rate and batch; reusing that seed may favor the selection.

Setting	PG	DQN
Method; evaluation	RTG; stochastic	DQN; greedy
Learning rate; batch	0.003; 2000 fresh steps	0.001; batch 64
Replay / update settings	Fresh on-policy batches	Capacity 100,000; warm-up 1000; target sync every 500 updates

Observations after 200,000 steps are excluded. Linear interpolation uses the shared supported endpoint of **190,831 steps**, without extrapolation; AUC uses a common 10,000-step grid including that endpoint.



CartPole evaluation returns: PG and DQN

Discussion

Which algorithm converges faster, and what might explain the difference?

DQN confirms three consecutive evaluation means of at least 475 in 3/3 seeds, with median 60,000 steps, versus 0/3 for PG; its AUC is also higher (298.8 ± 36.5 versus 248.1 ± 27.5). Replay reuse and frequent updates may improve sample efficiency compared with fresh on-policy batches. Different selected parameters and greedy versus stochastic evaluation prevent isolating the update rule, and threshold confirmation does not guarantee later stability.

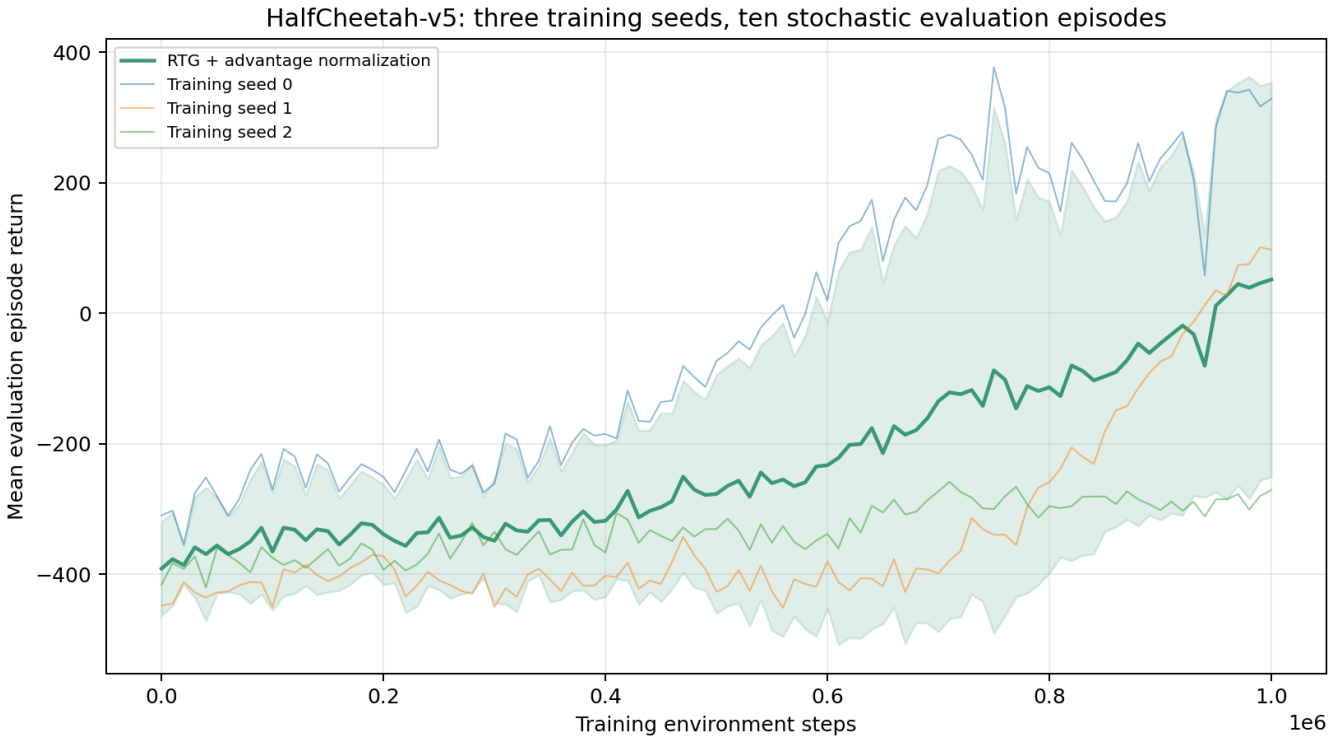
Task 2 - Question 4: HalfCheetah controller

Question

Train and tune a PG controller for HalfCheetah, visualize progress with TensorBoard, submit its learning curve and inspect the trained agent in the video view.

Submit

The required plot shows evaluation return versus environment steps. The selected controller uses RTG with advantage normalization, learning rate 0.001, batch 5000, gamma 0.99 and a 64 x 2 ReLU Gaussian actor. There is no critic or gradient clipping; each seed completes 1,000,000 steps. Normalization changes the finite-batch estimator and is an extension beyond the core Q1/Q2 comparison; exploratory seed-0 pilot selection can favor that seed.

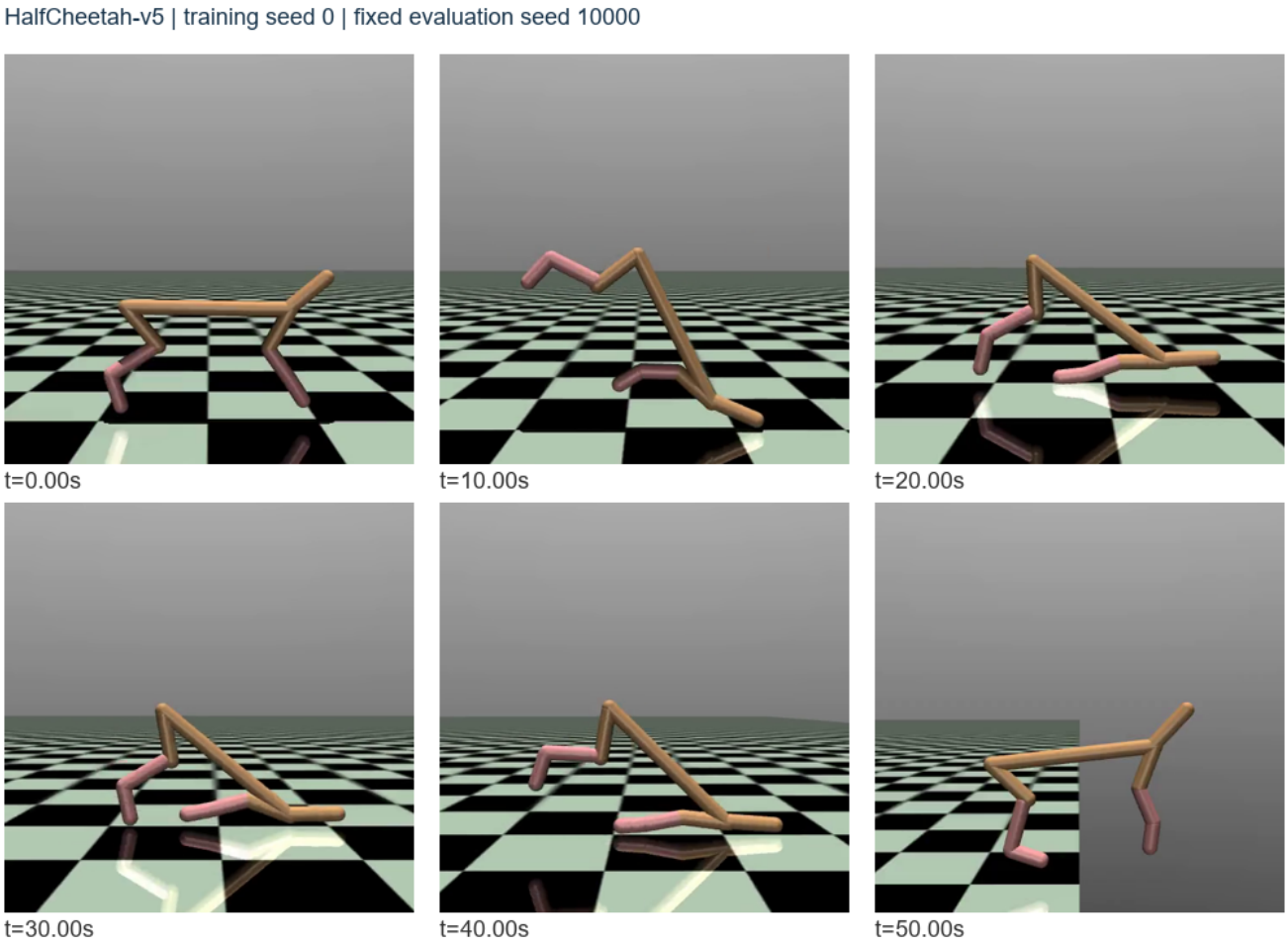


HalfCheetah evaluation learning curves

Training seed	Initial return	Final return	Last-five mean
0	-309.9	328.7	333.5
1	-448.3	97.0	74.8
2	-416.3	-271.1	-282.9

Observations

Mean final return is 51.5 ± 302.5, improved from -391.5 ± 72.4 initially, but seed 2 remains negative; this does not establish reliable control. TensorBoard scalars and a video preview were exported after training and checked at event-file level; no browser or Video-tab inspection is claimed.



Sampled frames from the fixed seed-0 episode

The decoded, sampled frames show substantial posture changes. The independently reproduced seed-0 episode returns 364.09, travels 39.96 m in 50 seconds, and has absolute torso pitch above 45 degrees in 34.1% of recorded states; this is a descriptive threshold, not a solved rule. Its checkpoint still has one negative episode among ten evaluations, with worst return -441.05; decoding and sampled frames do not establish full-playback quality or uniformly reliable behavior.

Workshop 5: Actor Critic - Final Report

Authors: Zhengnan Cao, Yuchen Yang

This report answers Task 1 and all four Task 2 questions using 17 formal conditions. Each condition uses the preselected training seed 0: AC/GAE requests 500,000 steps and SAC runs for 1,000,000 steps. Every 10,000 training steps, the stochastic policy is evaluated for ten fixed-seed episodes (10000-10009), independently of training. Evaluation steps are excluded from the horizontal axis. The same SAC baseline is reused across questions.

Tables distinguish the final evaluation from the mean of the last five checkpoints. Tail SD is the population standard deviation across those five checkpoint means, not across training seeds. AUC is the time-averaged return over the recorded interval (10k-1M for SAC); first return 1000 is a descriptive speed diagnostic, not a convergence criterion. SAC training scalars are counted 1000-update window means, including the shorter final window; plots add no smoothing. One seed supports descriptive comparisons only.

Late scalar summaries (alpha, entropy, Q, targets and critic MSE) use logged SAC windows ending in the final 50k-step region (950k-1M), weighted by each window's actual update count, window_updates.

Task 1 - Question 1: HalfCheetah environment and policies

Question

What are the state space, action space and reward structure? Are they finite or infinite, discrete or continuous? Is the environment deterministic or stochastic, and are the AC and SAC policies deterministic or stochastic?

Answer

Component	Description and classification
State / observation	17 continuous coordinates: eight generalized positions (global horizontal position excluded) and nine velocities. The observation Box is unbounded and contains infinitely many values.
Action	Six continuous joint-torque controls in [-1, 1]. Bounded but with infinitely many possible values.
Reward	Continuous scalar: forward velocity minus 0.1 times the sum of squared actions. Its set of possible values is infinite.

With default settings, forward velocity is (x_next - x)/0.05. Given the complete simulator state and action, MuJoCo physics is deterministic. Reset adds uniform position noise and Gaussian velocity noise with scale 0.1, so episodes start stochastically. HalfCheetah has no task termination by default; TimeLimit truncates episodes at 1000 steps.

Both policies are stochastic during training and evaluation. AC samples a Gaussian action, clipping only the value sent to the environment; its log probability uses the original sample. SAC samples a Gaussian and applies tanh. Fixed evaluation seeds make sampled episodes reproducible without making the policies deterministic.

Task 1 - Question 2: AC/GAE training on HalfCheetah

Question

Train AC with GAE (lambda=0.99) on HalfCheetah-v5. Log progress with TensorBoard, submit evaluation returns and actor loss versus environment steps, and comment on training success.

Submit

Configuration: 500k steps, gamma=0.99, Actor/Value LR=0.001, two 64-unit hidden layers, batches of at least 1000 steps and normalized advantages. GAE recurses backward within each episode. Only true termination suppresses bootstrap; either termination or truncation stops the recursion. The value target is advantage plus the current value estimate.

$$\delta_t = r_t + \gamma(1 - \text{terminated}_t)V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - \text{done}_t)\hat{A}_{t+1}, \qquad \hat{R}_t = \hat{A}_t + V(s_t)$$

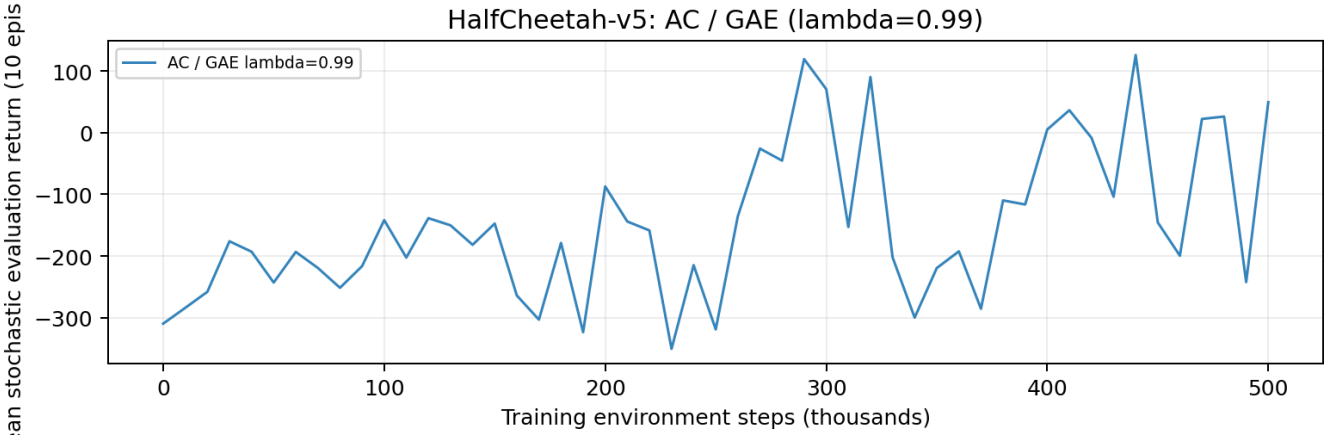


Figure 1. AC/GAE evaluation returns versus training environment steps.

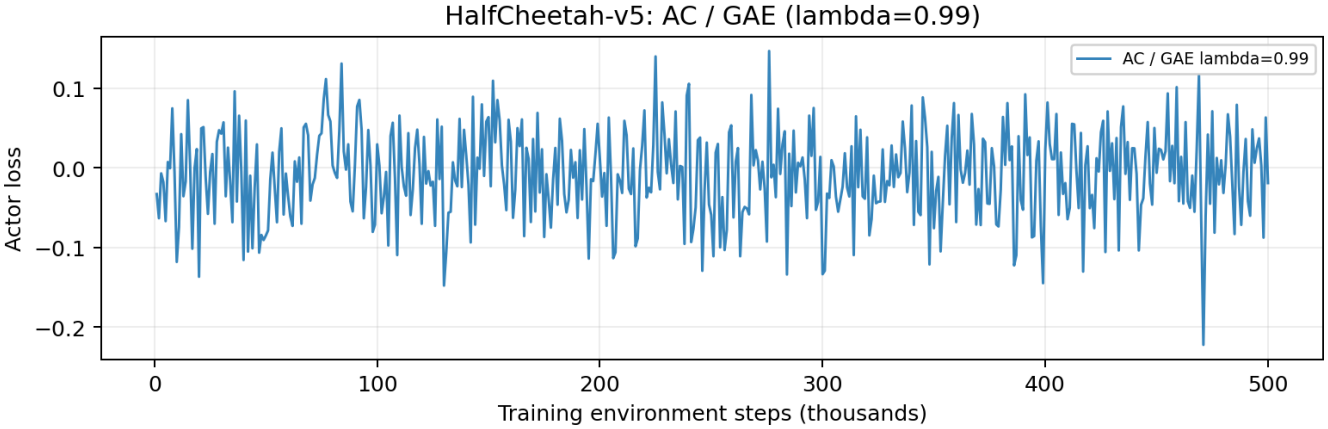


Figure 2. AC actor loss, recorded once per update batch.

Discussion

Does the learning curve suggest successful training?

Return improves from -309.9 at the first checkpoint to 49.7 at the last one, but the last-five mean is -68.7 and tail SD is 125.5. The frequent drops and non-positive late mean do not demonstrate stable successful learning. The positive final checkpoint alone is insufficient; actor-loss sign or decrease is not a success criterion.

Task 1 - Question 3: SAC training on HalfCheetah

Question

Train SAC on HalfCheetah-v5, visualize progress with TensorBoard and submit evaluation-return and actor-loss curves. Check positive return by 50k steps and continued improvement beyond 500k steps.

Submit

Baseline: gamma=0.99, Actor/Critic LR=1e-4, two 256-unit hidden layers, initial alpha=0.2 with automatic temperature tuning (target entropy -6, temperature LR=3e-4), tau=0.005, replay capacity 1M, batch=256 and warm-up=5000. One update follows each transition after warm-up. The target uses the smaller target critic and an entropy term; the actor uses reparameterized actions with the tanh log-probability correction.

$$y = r + \gamma(1 - \text{terminated})[\min_{i=1,2} \bar{Q}_i(s\prime, a\prime) - \alpha \log \pi_\theta(a\prime \mid s\prime)]$$

$$J_\pi = \mathbb{E}_{s \sim \mathcal{D},\, a \sim \pi_\theta(\cdot \mid s)}[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_i(s, a)]$$

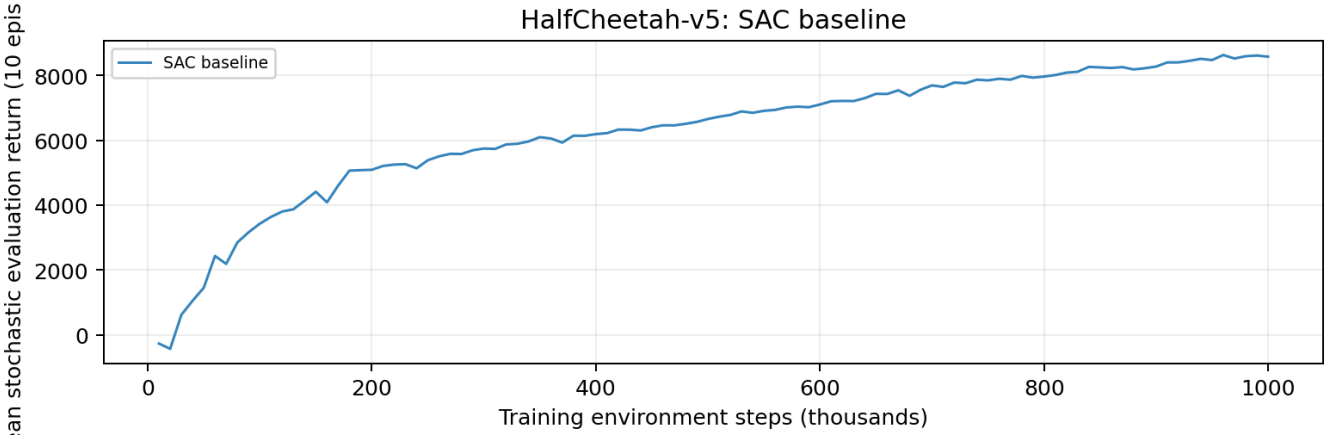


Figure 3. SAC baseline evaluation returns.

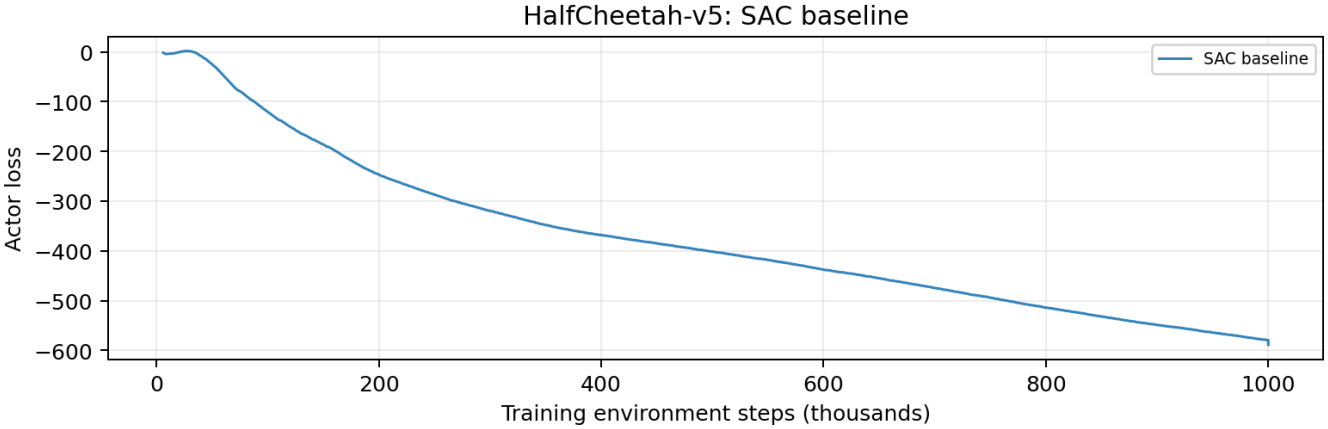


Figure 4. SAC actor loss; counted update-window means.

Discussion

Does SAC meet the expected learning behavior?

Yes, in the observed seed-0 run. Return first becomes positive at 30,000 steps and is 1,453.9 at 50k. It reaches 6,657.2 at 500k and 8,578.0 at 1M; the last-five mean is 8,587.7. Mean return rises from 6,509.3 over 450k-500k to 8,568.7 over 950k-1M. This supports useful learning and further late improvement, while not proving that no temporary plateau occurs or that the result generalizes across training seeds.

TensorBoard logging was enabled. The submitted plots reconstruct the CSV scalar records, whose values were checked against the event files; this verification does not constitute actual browser inspection.

Task 2 - Question 1: Effect of GAE lambda

Question

Compare at least three lambda values in [0.9, 0.99] on InvertedPendulum-v5. Submit evaluation-return curves and discuss observed behavior, the lambda=0 and lambda=1 estimators, and the bias-variance trade-off.

Submit

Values are 0.90, 0.95 and 0.99. Other AC settings match Task 1; each run requests 500k steps and finishes complete episodes, explaining the small budget overshoots below.

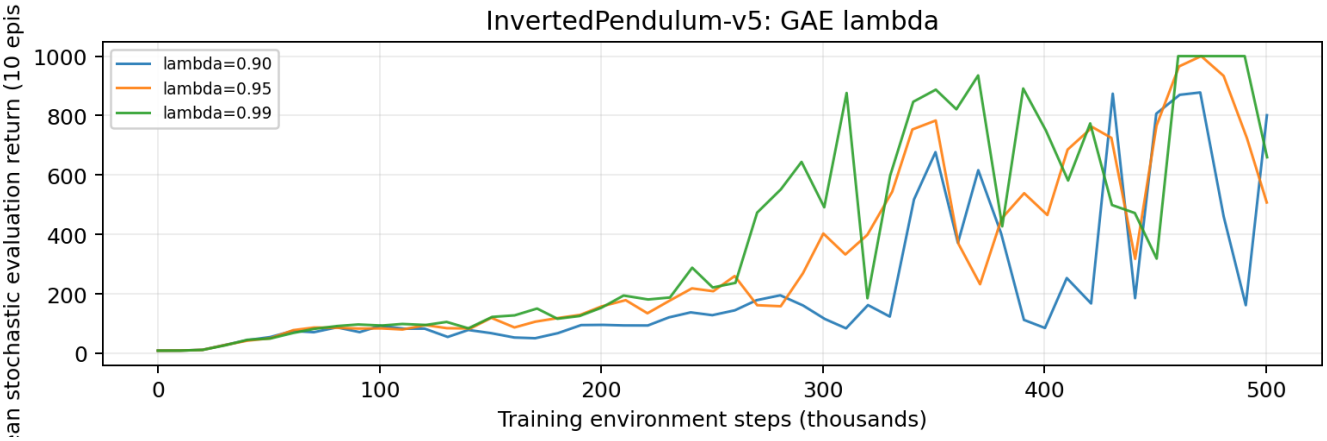


Figure 5. InvertedPendulum evaluation returns for all three GAE values.

Lambda	Actual steps	Final return	Last-five mean	Tail SD
0.90	500,295	801.1	634.1	281.4
0.95	500,233	507.5	826.0	186.1
0.99	500,376	659.7	931.9	136.1

Discussion

What behavior is observed for different lambda values?

Lambda=0.99 has the highest last-five mean (931.9), followed by 0.95 (826.0) and 0.90 (634.1). The 0.95 and 0.99 runs first reach reference return 950 near the 460k evaluation; their few-hundred-step difference is smaller than the 10k evaluation interval. All three have late drops, so these results favor 0.99 for late performance without establishing stable convergence.

What do lambda=0 and lambda=1 correspond to?

Lambda=0 gives the one-step TD residual. Lambda=1 sums all discounted TD residuals: at a truly terminating episode boundary this telescopes to Monte Carlo return minus V(s); finite time-limit truncation retains a bootstrap term. These are theoretical endpoints; the experiments use only 0.90, 0.95 and 0.99.

How does lambda affect bias, variance and performance?

Smaller lambda relies more on the critic, usually reducing variance but increasing sensitivity to value-estimation bias. Larger lambda uses longer reward paths, usually reducing bootstrap bias while increasing variance. The better late returns at 0.99 are consistent with a useful trade-off here; return variation is not a direct measurement of gradient-estimator variance.

Task 2 - Question 2: Initial entropy temperature

Question

Compare initial alpha in [0.01, 1] with automatic temperature tuning and other settings fixed. Submit evaluation-return and actor-loss curves; discuss exploration, speed, stability, possible policy collapse and the entropy objective.

Submit

Initial values are 0.01, 0.2 and 1.0; the Task 1 SAC baseline supplies 0.2. These are initial temperatures, not constant entropy weights throughout training.

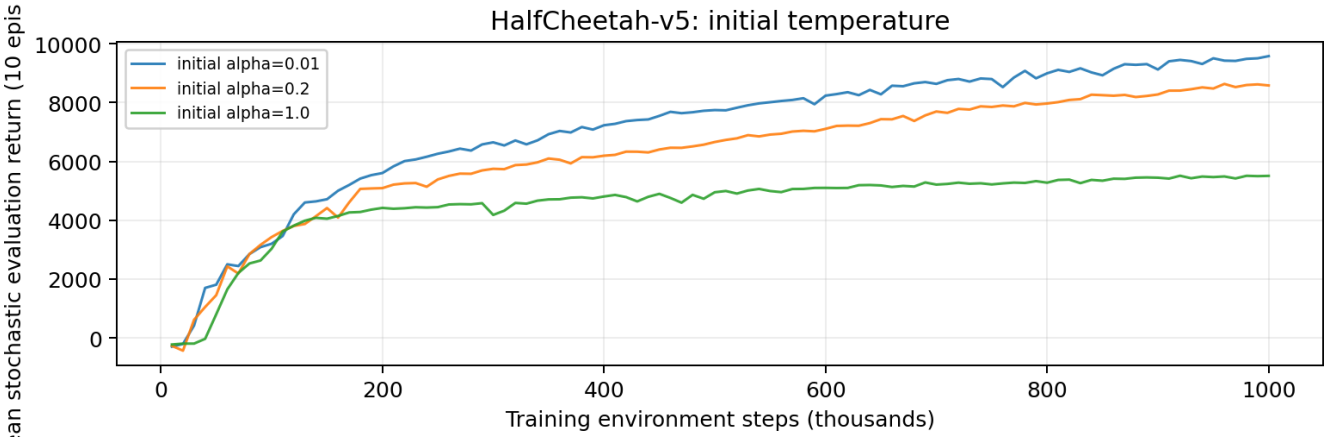


Figure 6. Evaluation returns for three initial temperatures.

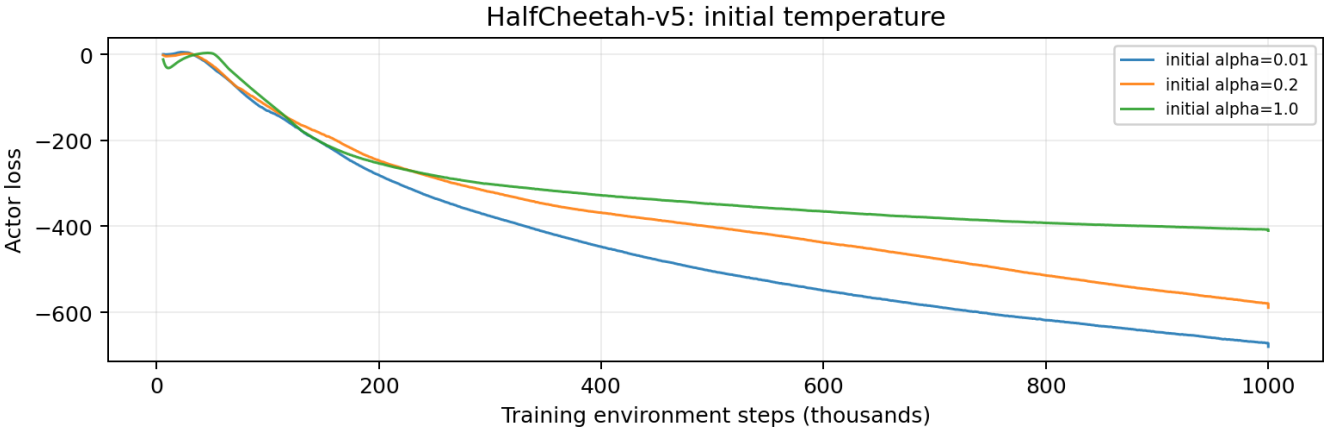


Figure 7. Actor loss for three initial temperatures.

Initial alpha	50k return	Last-five mean	Tail SD	Late alpha
0.01	1,809.2	9,478.2	59.0	0.246
0.2	1,453.9	8,587.7	37.1	0.263
1.0	803.7	5,483.9	33.3	0.137

Discussion

How do lower and higher initial temperatures affect exploration, speed and stability?

Higher alpha initially gives entropy a larger weight, encouraging exploration while potentially delaying exploitation. Here 0.01 and 0.2 first become positive at 30k, while 1.0 does so at 50k. Their last-five means are 9478.2, 8587.7 and 5483.9: the largest initial temperature learns more slowly and finishes lower in this seed. Automatic tuning later changes all three temperatures, but does not remove the influence of the early data-collection path.

For late stability, alpha=0.01 has tail SD 59.0, compared with 37.1 for 0.2: its higher return accompanies more checkpoint fluctuation. Alpha=1.0 has the smallest tail SD (33.3), but also the lowest return. Lower SD therefore does not establish a better policy; this measures local return variation, not gradient variance or across-seed stability.

Does a small alpha cause premature policy collapse?

It can weaken the exploration incentive in principle, but the 0.01 run attains the highest return of the three and its late entropy estimate is near the target -6. This run therefore does not support inevitable collapse at 0.01. The late entropy estimates of all three are near -6; negative differential entropy is valid for a continuous distribution and is not itself collapse. Action-coverage or multi-seed diagnostics would be needed for a stronger claim.

How does this relate to the SAC entropy objective?

The actor minimizes alpha*log pi - min(Q1,Q2), balancing reward-seeking against entropy. Automatic tuning targets entropy rather than maximizing return directly. Initial exploration and replay contents can produce different final returns even after entropy estimates become similar. Actor-loss magnitude combines the entropy and Q terms, so it cannot independently rank policy quality.

Task 2 - Question 3: Actor and critic learning rates

Question

Sweep actor and critic learning rates within [1e-5, 1e-2]. Submit evaluation returns, actor loss and both critic losses. Discuss each tested pair and identify the fastest useful learning without destabilization.

Submit

Five one-factor-at-a-time pairs test the low and high endpoints around the 1e-4/1e-4 baseline. This is not a full two-dimensional grid. All other settings and the 1M-step budget are fixed. Critic losses use a symlog axis (linear below 1) to show the near-zero and larger losses together.

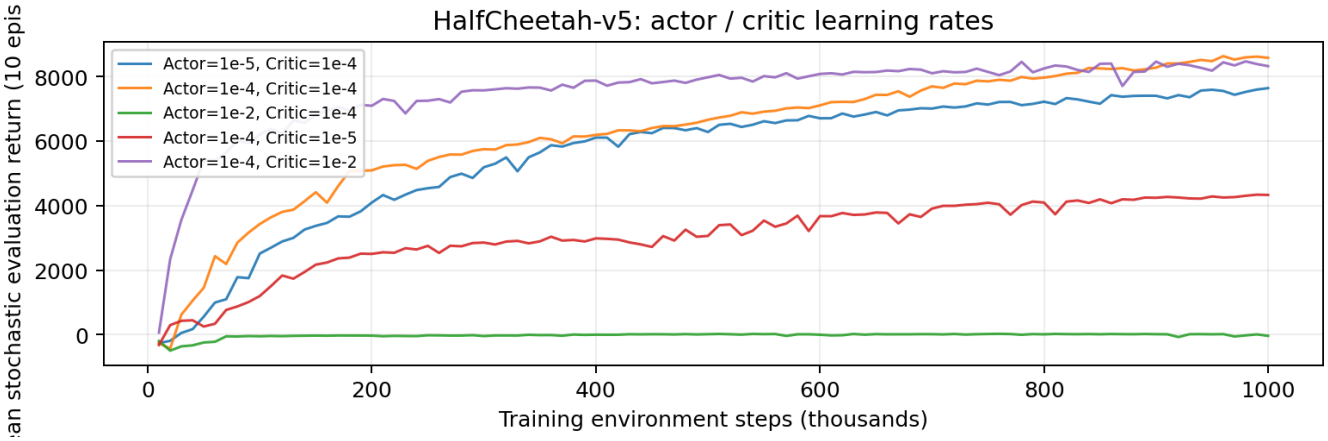


Figure 8. Learning-rate evaluation returns.

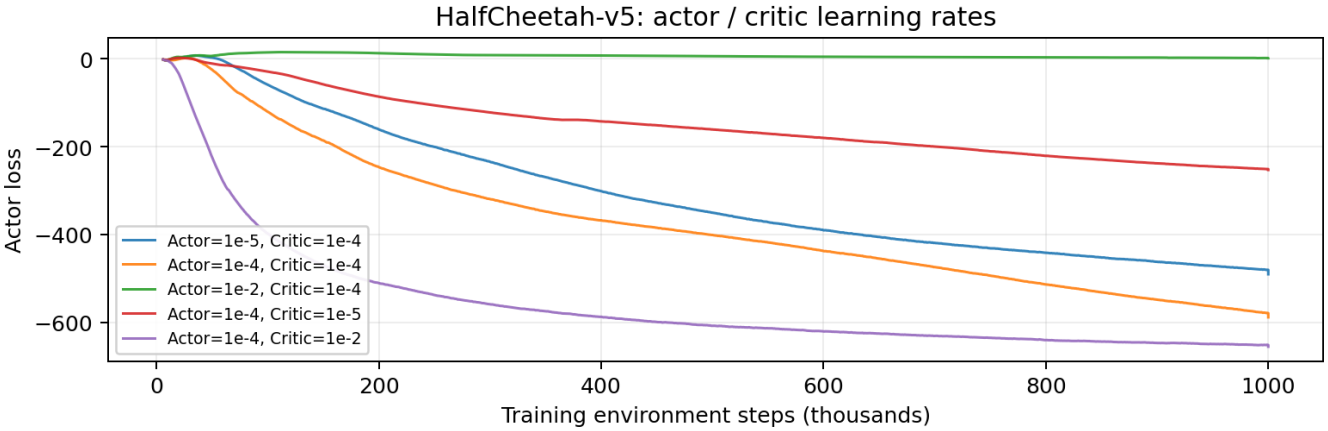


Figure 9. Learning-rate actor losses.

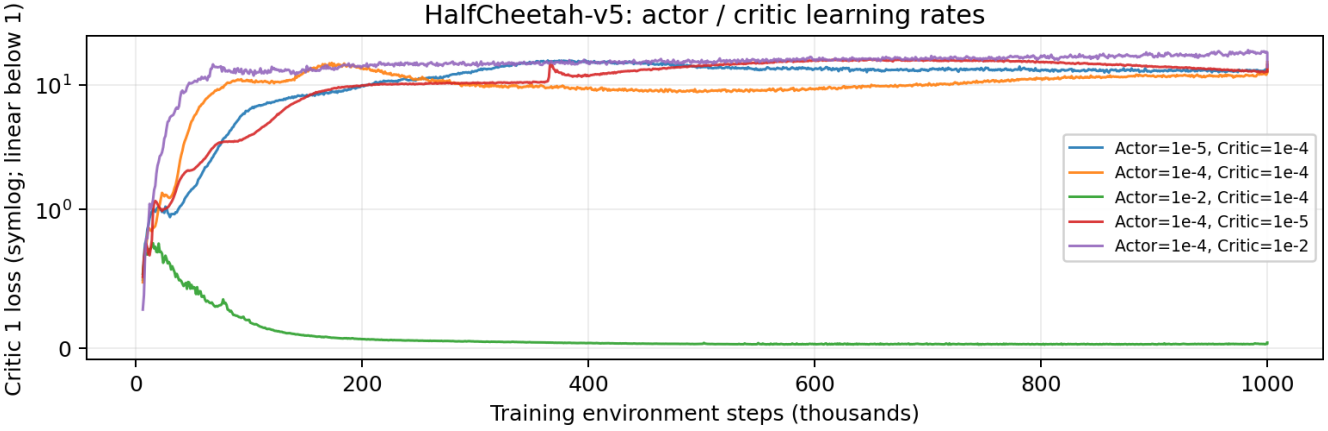


Figure 10. Learning-rate Critic 1 losses.

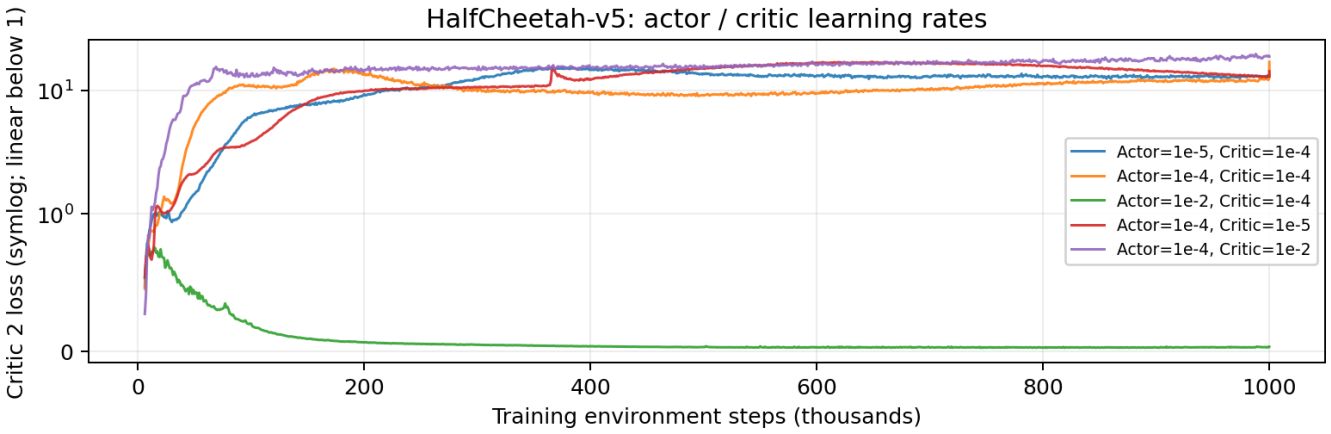


Figure 11. Learning-rate Critic 2 losses.

Actor LR	Critic LR	Last-five mean	Tail SD	AUC	First 1000
1e-5	1e-4	7,547.5	69.8	5,640.4	70,000
1e-4	1e-4	8,587.7	37.1	6,298.8	40,000
1e-2	1e-4	-18.5	27.2	-23.7	not reached
1e-4	1e-5	4,296.0	35.3	3,082.9	90,000
1e-4	1e-2	8,390.4	55.9	7,533.4	20,000

Discussion

Which pair learns fastest without destabilizing?

Actor=1e-4/Critic=1e-2 is the strongest speed candidate here: it first reaches return 1000 at 20k and has the highest AUC (7533.4), versus 40k and 6298.8 for the baseline. It completes 1M steps with finite losses and a last-five mean of 8390.4. The baseline has the higher late mean (8587.7) and lower tail SD (37.1 versus 55.9). Thus the high critic rate improves early sample efficiency with useful late performance, while the baseline is better on the late-return measure. Neither establishes strict or cross-seed convergence.

What happens in each tested case?

Actor=1e-5, Critic=1e-4. Reaching return 1000 takes 70k steps, and the last-five mean is 7547.5. Relative to the baseline, the smaller actor step delays improvement; the late curve still rises, so this is slower learning rather than numerical divergence.

Actor=1e-4, Critic=1e-4. This baseline has the highest last-five mean among these pairs. Its mean return increases by 2059.4 between the 450k-500k and 950k-1M windows, so it remains useful late in training without evidence that it has completely stabilized.

Actor=1e-2, Critic=1e-4. The last-five mean is -18.5 despite finite late Critic 1/2 MSEs of 0.029/0.028. Large policy updates can chase inaccurate value estimates; these measurements are consistent with ineffective control, not a measured gradient explosion. Fitting a low-reward replay distribution can give a small MSE without a good policy. This unfavorable result is retained.

Actor=1e-4, Critic=1e-5. It needs 90k steps to reach return 1000 and finishes with last-five mean 4296.0. This slower return growth is consistent with slow critic adaptation. Its small tail SD (35.3) does not offset the low return or prove lower gradient variance.

Actor=1e-4, Critic=1e-2. Fastest early growth, slightly lower late return than the baseline. Its late critic MSEs are 17.955/18.627, versus 11.847/12.062 for the baseline; finite MSE alone cannot rank policies with different replay distributions.

Task 2 - Question 4: Replay size, batch size and learning start

Question

Compare replay capacities {100k, 500k, 1M}, batches in [32, 512] and learning starts {1k, 5k, 10k}. Submit evaluation returns, Q values and target values for each; discuss adaptation, overfitting, variance, stability and sample efficiency.

Submit

Each factor changes alone from replay=1M, batch=256 and start=5k. Batches are 32, 256 and 512; all conditions run for 1M environment steps. Q is the mean min(Q1,Q2) at replay states with newly sampled current-policy actions. Target is the mean entropy-regularized bootstrap target for replay transitions. These use different actions and constructions; their mean difference is not critic MSE.

Replay capacity

Setting	Last-five mean	Tail SD	AUC	Late Q	Late target
Replay=100k	9,947.3	397.2	7,481.9	782.2	782.8
Replay=500k	9,420.5	63.2	6,502.6	714.5	714.0
Replay=1M	8,587.7	37.1	6,298.8	573.7	572.7

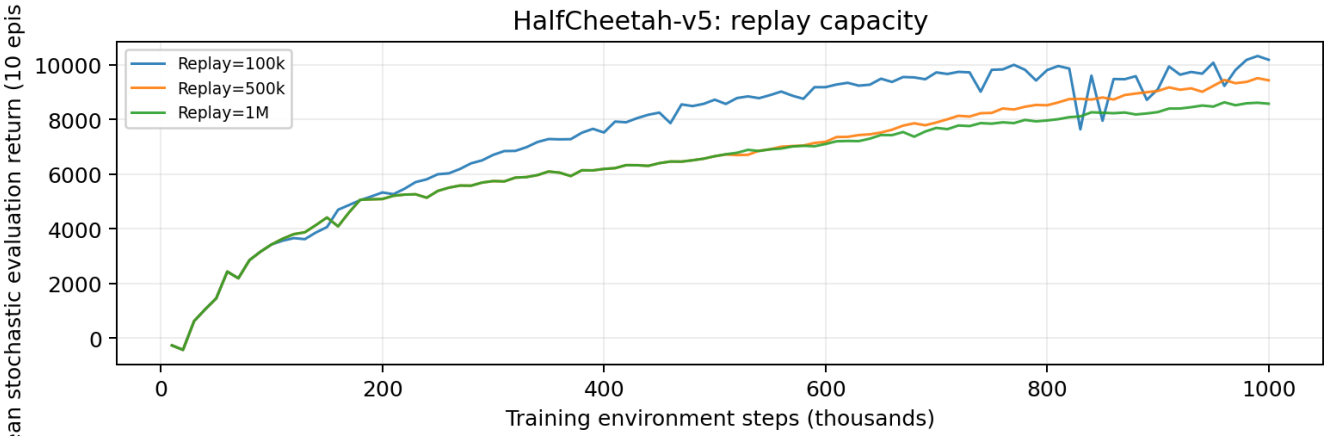


Figure 12. Replay capacity: evaluation returns.

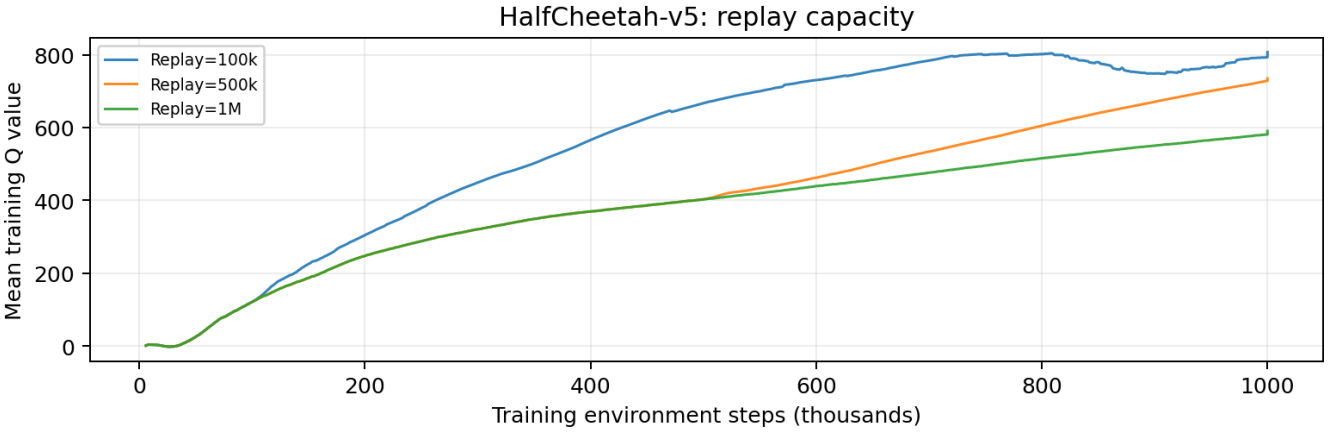


Figure 13. Replay capacity: mean training Q values.

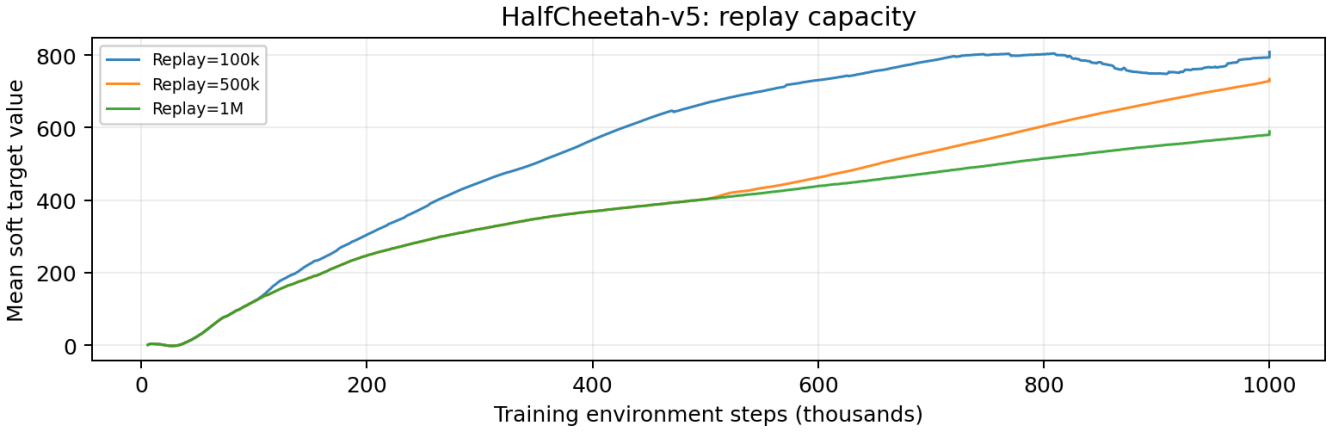


Figure 14. Replay capacity: mean soft targets.

Batch size

Setting	Last-five mean	Tail SD	AUC	Late Q	Late target
Batch=32	5,584.7	10.6	4,314.5	350.4	350.2
Batch=256	8,587.7	37.1	6,298.8	573.7	572.7
Batch=512	7,182.7	51.1	5,575.4	477.0	476.6

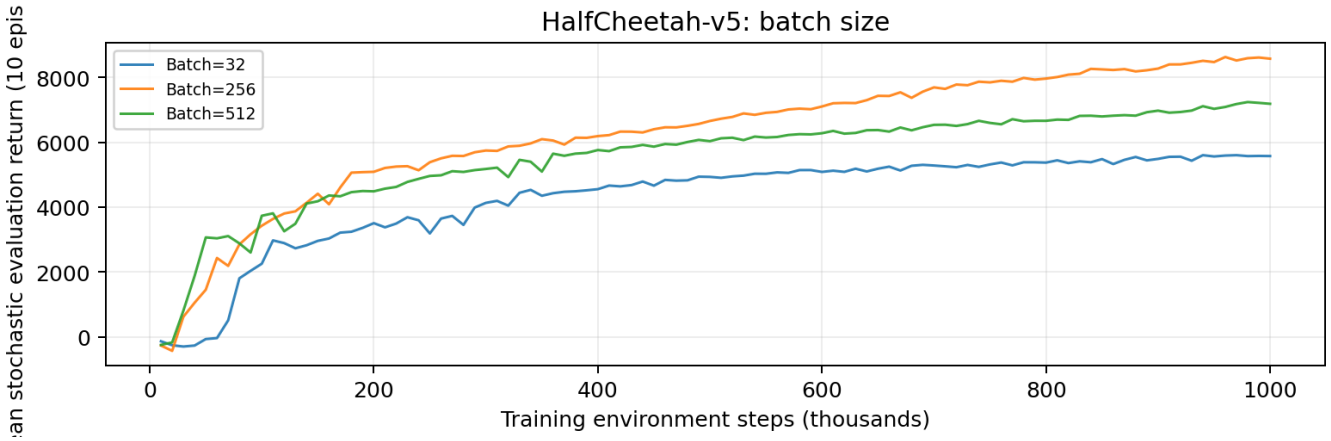


Figure 15. Batch size: evaluation returns.

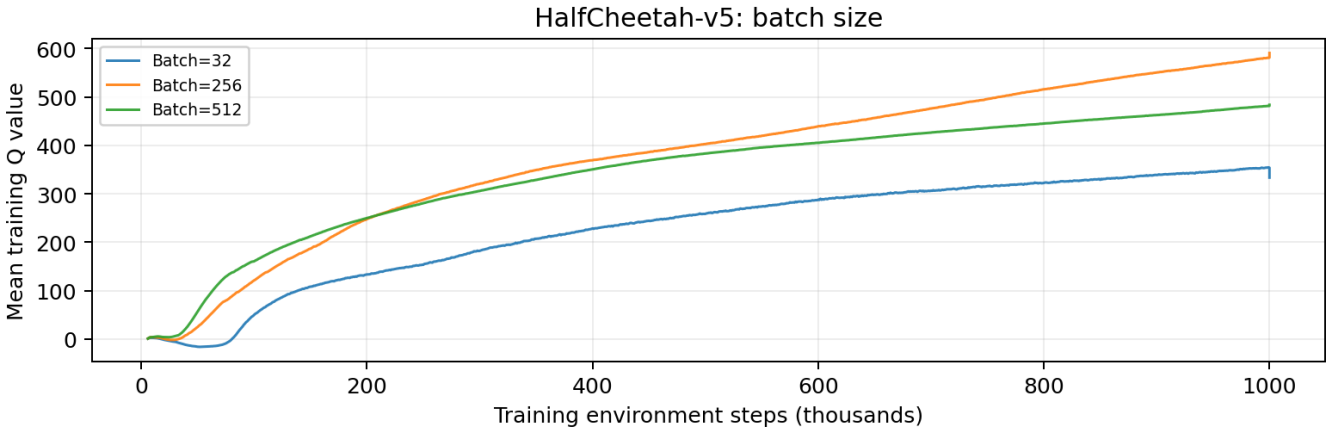


Figure 16. Batch size: mean training Q values.

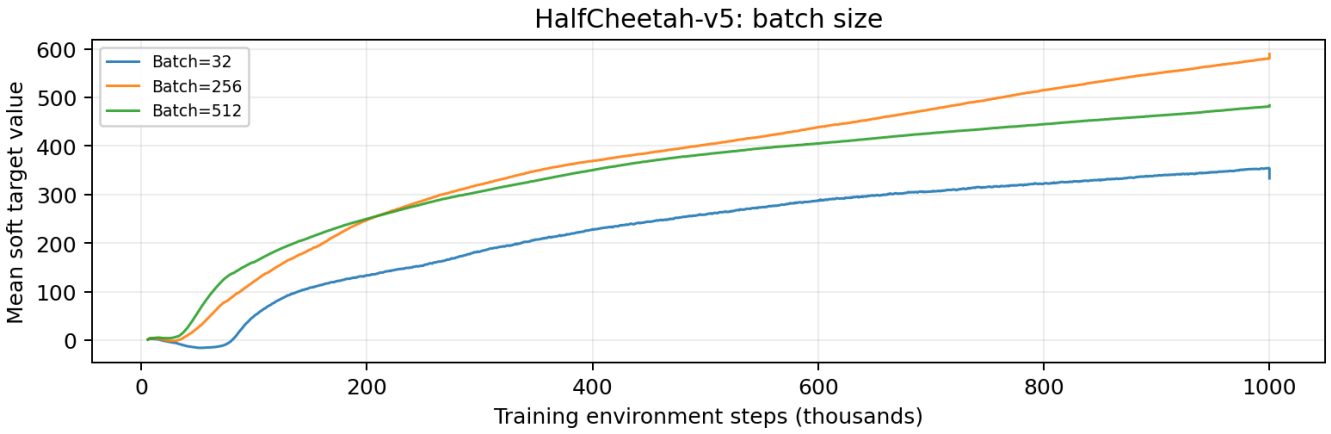


Figure 17. Batch size: mean soft targets.

Learning start

Setting	Last-five mean	Tail SD	AUC	Late Q	Late target
Start=1k	5,420.6	23.4	4,470.7	386.3	386.1
Start=5k	8,587.7	37.1	6,298.8	573.7	572.7
Start=10k	8,181.0	59.1	6,034.1	577.8	576.9

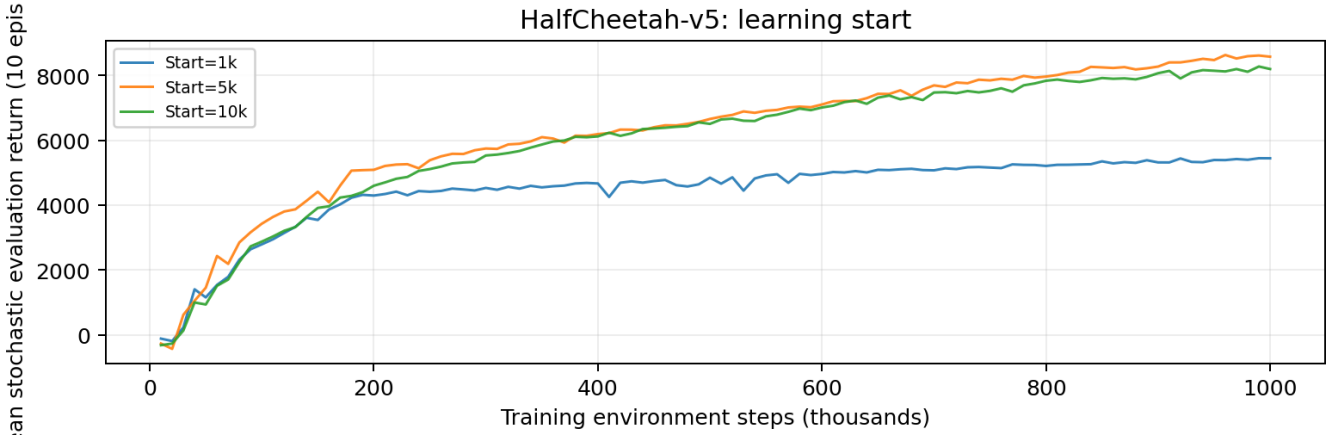


Figure 18. Learning start: evaluation returns.

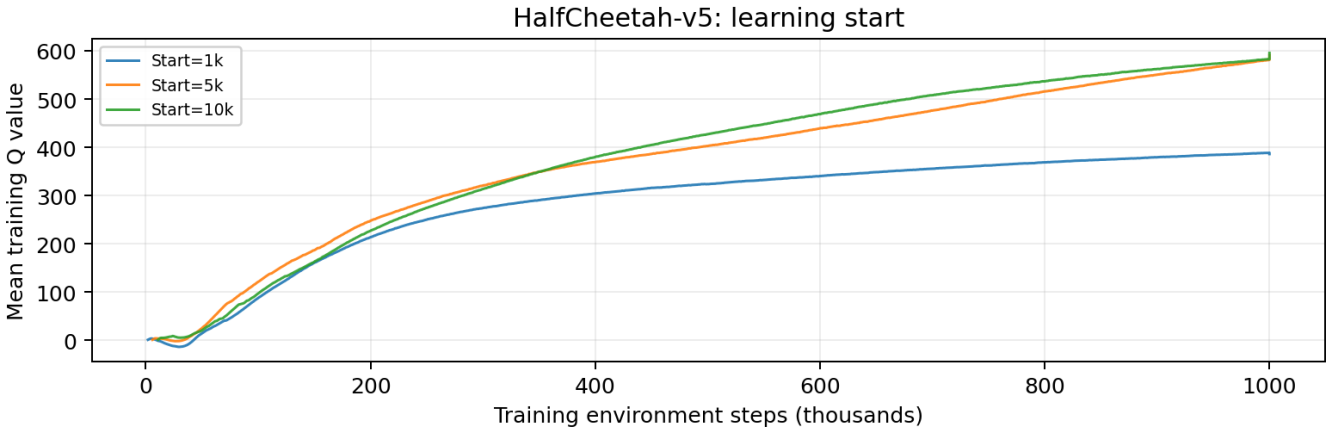


Figure 19. Learning start: mean training Q values.

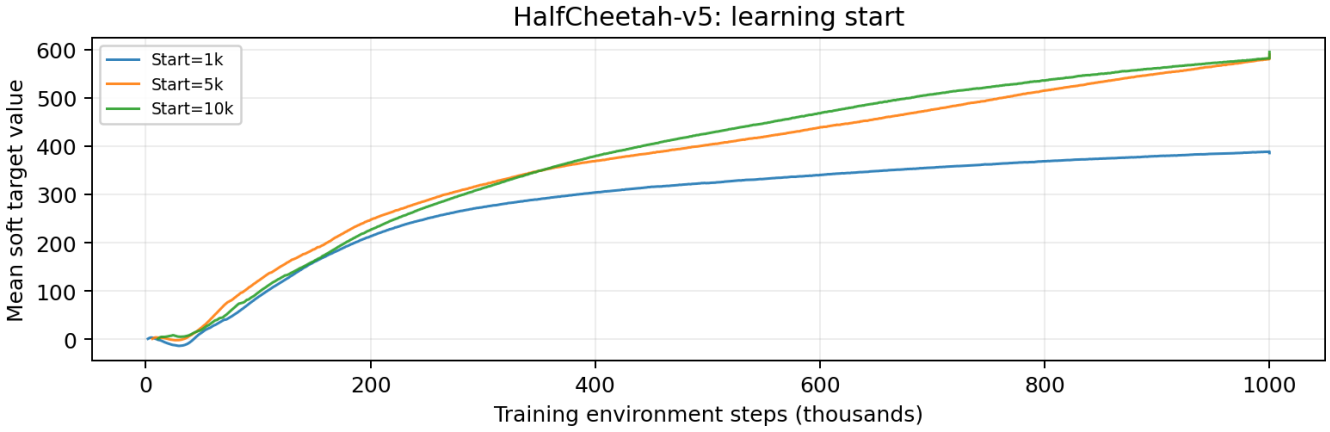


Figure 20. Learning start: mean soft targets.

Discussion

Does a smaller buffer adapt faster but overfit, while a larger buffer stabilizes learning?

Replay=100k achieves the highest last-five return (9947.3), followed by 500k (9420.5) and 1M (8587.7), but their tail SDs are 397.2, 63.2 and 37.1. Smaller replay can replace old-policy data faster, consistent here with higher return and more late fluctuation. This is an adaptation/stability trade-off, not evidence establishing overfitting: no held-out generalization experiment was run.

The 100k and 500k runs exactly match the baseline at their 10 and 50 shared evaluation checkpoints through 100k and 500k steps, respectively, before different eviction behavior starts. All three first reach return 1000 at 40k, so that early threshold cannot distinguish capacity effects. The runs are independent executions with a common fixed-seed prefix; post-eviction behavior is the relevant comparison.

Do larger batches reduce variance and improve stability or sample efficiency?

Batch=256 has the highest late mean (8587.7) and AUC (6298.8) of the three. Batch=512 has better return at 50k (3066.9 versus 1453.9), but finishes lower (7182.7), so its early advantage does not persist. Batch=32 reaches return 1000 at 80k rather than 40k and has last-five mean 5584.7. Its smaller tail SD (10.6) describes a flatter, lower-return trajectory; it is not evidence of smaller gradient variance.

All batches make 995001 updates. Sampling 32, 256 or 512 replay items per update changes replay work, not the number of new environment interactions. Observed process times are 35.7, 40.7 and 44.0 minutes; these include startup and evaluation, so they are separate from environment-sample efficiency and are not pure GPU timings.

Does longer warm-up reduce variance and improve stability?

Not monotonically in this run. Starts 1k/5k/10k give last-five returns 5420.6/8587.7/8181.0 and tail SDs 23.4/37.1/59.1. The 1k and 5k settings reach return 1000 at 40k, while 10k needs 60k; 5k also has the highest AUC. Earlier learning therefore does not automatically improve sample efficiency, and the longest warm-up is not the most stable late policy here.

Longer warm-up increases random experience before learning but delays updates. The update counts are 999001, 995001 and 990001. Differences reflect both the initial data and update timing; no extra ablation isolates these causes. Q and target means stay on comparable scales, but such averages cannot prove accurate value calibration or reduced gradient variance.

Supplementary temperature diagnostics

The following curves support Task 2 Question 2. Late temperature and entropy summaries weight each window by its actual update count, so the final short window is not over-weighted.

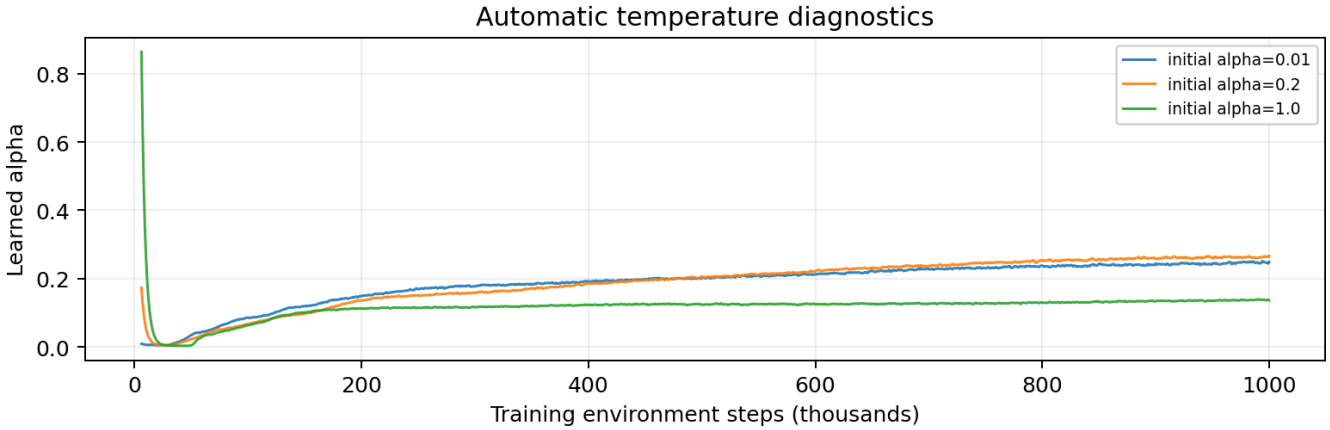


Figure 21. Automatically tuned temperature from each initial alpha.

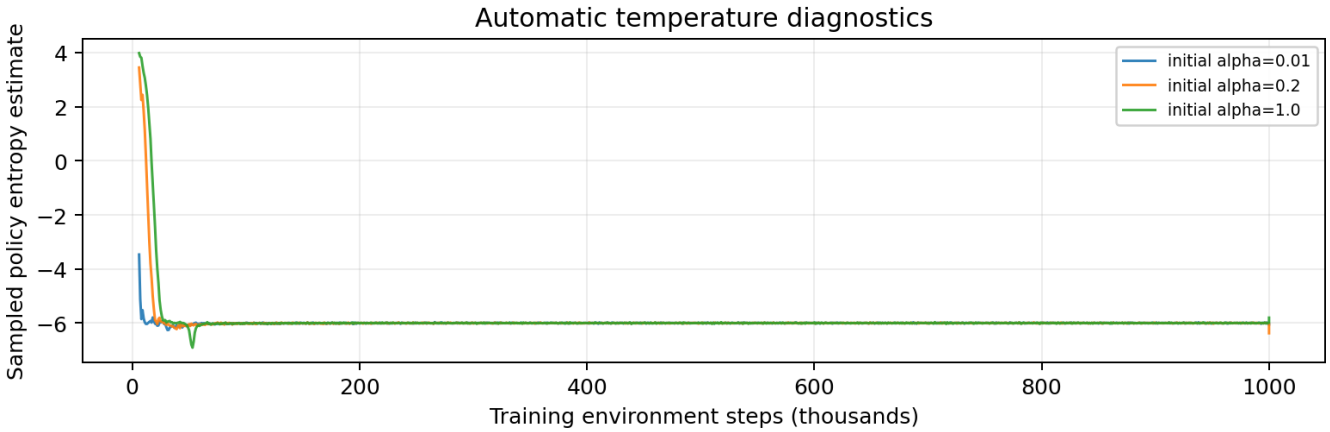


Figure 22. Sampled policy differential entropy; target -6.

References and reproducibility

Provided ELEN90098 Workshop 5 handout, pp. 5-7 (tasks), pp. 2-4 (environment and algorithm definitions). Environment details were checked against the installed Gymnasium 1.3.0 implementation. Runtime, full configurations, frozen source, CSVs, TensorBoard events and checkpoints are retained under evidence; the approved run map and numerical summary are evidence/planning/approved_default_steps_matrix.json and evidence/analysis/formal_results_summary.json. Implementation optimizations preserve one SAC update per transition and include Chinese comments; verification details are separate from the answers above.